MINI PROJECT DPW

Dosen Pengampu: Isbat Uzzin Nadhori S.Kom., M.T



Disusun untuk Memenuhi Tugas

Mata Kuliah:

PRAKTIKUM DATA PROCESSING WORKFLOW

Oleh:

Sheila Kamila Zahra

3324600034

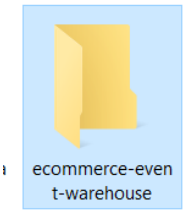
Sains Data Terapan B

PROGRAM STUDI D4 SAINS DATA TERAPAN

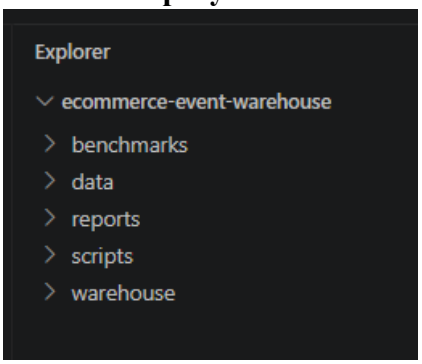
DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

1. **buat folder baru**



2. **buat folder proyek**



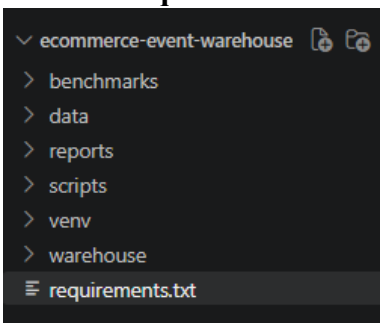
3. **hasil pemeriksaan lingkungan**

```
PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> python --version
Python 3.10.0
PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> py --list
 -V:3.13 *        Python 3.13 (64-bit)
 -V:3.10          Python 3.10 (64-bit)
PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> docker --version
Docker version 28.1.1, build 4eba377
PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> docker compose version
Docker Compose version v2.35.1-desktop.1
PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse>
```

4. **buat environment**

```
PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> python -m venv venv
PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> .\venv\Scripts\Activate.ps1
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse>
```

5. **buat file requirements.txt**



```
Welcome   requirements.txt
requirements.txt
1  requests>=2.31,<3
2  pandas>=2.1,<3
3  pyarrow>=14,<24
4  pydantic>=2.6,<3
5  polars>=0.20,<2
6  duckdb>=1.1,<2
7  confluent-kafka>=2.5,<3
8  dbt-duckdb>=1.8,<2
```

6. **install semua library**

```
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> python -m pip install -r requirements.txt
Collecting requests<3,>=2.31
  Using cached requests-2.34.2-py3-none-any.whl (73 kB)
Collecting pandas<3,>=2.1
  Using cached pandas-2.3.3-cp310-cp310-win_amd64.whl (11.3 MB)
Collecting pyarrow<24,>=14
  Downloading pyarrow-23.0.1-cp310-cp310-win_amd64.whl (27.5 MB)
     |████████████████████████████████| 27.5 MB 3.3 MB/s
Collecting pydantic<3,>=2.6
  Downloading pydantic-2.14.1-py3-none-any.whl (488 kB)
     |████████████████████████████████| 488 kB 6.4 MB/s
Collecting polars<2,>=0.20
  Downloading polars-1.44.2-py3-none-any.whl (865 kB)
     |████████████████████████████████| 865 kB 1.3 MB/s
Collecting duckdb<2,>=1.1
  Downloading duckdb-1.5.6-cp310-cp310-win_amd64.whl (13.2 MB)
     |████████████████████████████████| 13.2 MB 3.3 MB/s
Collecting confluent-kafka<3,>=2.5
  Using cached confluent_kafka-2.16.0-cp310-cp310-win_amd64.whl (4.7 MB)
Collecting dbt-duckdb<2,>=1.8
  Downloading dbt_duckdb-1.11.0-py3-none-any.whl (91 kB)
     |████████████████████████████████| 91 kB 1.9 MB/s
```

7. **pastikan dataset sudah ada**

```
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> Get-ChildItem .\data\source


    Directory: C:\Users\LEGION\Downloads\ecommerce-event-warehouse\data\source


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        10/25/2026  12:39 PM           2398 orders.csv


(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse>
```

8. **upgrade pip**

```
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> python -m pip install --upgrade pip
Requirement already satisfied: pip in c:\users\legion\downloads\ecommerce-event-warehouse\venv\lib\site-packages (21.2.3)
Collecting pip
  Downloading pip-26.2.2-py3-none-any.whl (1.8 MB)
     |████████████████████████████████| 1.8 MB 595 kB/s
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 21.2.3
    Uninstalling pip-21.2.3:
      Successfully uninstalled pip-21.2.3
Successfully installed pip-26.2.2
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse>
```

9. **periksa isi dataset**

```
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> python -c "import pandas as pd; df=pd.read_csv('data/source/orders.csv'); print('Ukuran dataset:', df.shape); print('\nKolom:', df.columns.tolist()); print('\nTipe data:\n', df.dtypes); print('\nLima baris pertama:\n', df.head().to_string()); print('\nJumlah nilai kosong:\n', df.isna().sum())"
Ukuran dataset: (36, 9)

Kolom: ['order_id', 'customer_id', 'order_ts', 'product_category', 'quantity', 'unit_price', 'discount_pct', 'payment_method', 'status']

Tipe data:
 order_id            object
customer_id          object
order_ts             object
product_category     object
quantity              int64
unit_price          float64
discount_pct        float64
payment_method       object
status               object
dtype: object

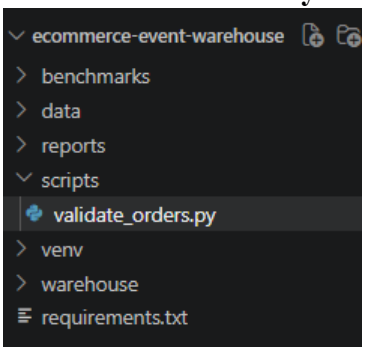
Lima baris pertama:
   order_id customer_id            order_ts product_category  quantity  unit_price  discount_pct payment_method     status
0    O1001        C001  2026-09-01T09:15:00Z      Electronics         1      250.00          0.05           card       paid
1    O1002        C002  2026-09-01T10:10:00Z             Home         2       45.50          0.00       transfer    shipped
2    O1003        C003  2026-09-01T11:05:00Z            Books         3       12.99          0.10        ewallet       paid
3    O1004        C001  2026-09-02T08:45:00Z      Electronics         1       89.90          0.15           card       paid
4    O1005        C004  2026-09-02T12:00:00Z          Fashion         2       39.00          0.20        ewallet  cancelled

Jumlah nilai kosong:
 order_id           0
customer_id         1
order_ts            0
product_category    0
quantity            0
unit_price          0
discount_pct        0
payment_method      0
status              0
dtype: int64
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse>
```

10. periksa apakah ada cacat

```
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> python -c "import pandas as pd; df=pd.read_csv('data/source/orders.csv'); print(df.to_string(index=False))"
order_id customer_id            order_ts product_category  quantity  unit_price  discount_pct payment_method    status
   O1001        C001 2026-09-01T09:15:00Z      Electronics         1      250.00          0.05           card      paid
   O1002        C002 2026-09-01T10:10:00Z             Home         2       45.50          0.00       transfer   shipped
   O1003        C003 2026-09-01T11:05:00Z            Books         3       12.99          0.10        ewallet      paid
   O1004        C001 2026-09-02T08:45:00Z      Electronics         1       89.90          0.15           card      paid
   O1005        C004 2026-09-02T12:00:00Z          Fashion         2       39.00          0.20        ewallet cancelled
   O1006        C005 2026-09-02T13:30:00Z             Home         1      120.00          0.00       transfer   shipped
   O1007        C006 2026-09-03T07:20:00Z            Books         5        8.50          0.00           card      paid
   O1008        C002 2026-09-03T09:40:00Z          Fashion         1       75.00          0.10        ewallet      paid
   O1009        C007 2026-09-03T15:10:00Z      Electronics         1      499.00          0.05           card  refunded
   O1010        C008 2026-09-04T10:25:00Z             Home         4       15.75          0.00       transfer      paid
   O1011        C009 2026-09-04T12:40:00Z            Books         2       18.00          0.05           card   shipped
   O1012        C010 2026-09-04T16:00:00Z          Fashion         3       22.00          0.10        ewallet      paid
   O1013        C003 2026-09-05T08:15:00Z      Electronics         1      129.00          0.00           card      paid
   O1014        C011 2026-09-05T09:55:00Z             Home         2       64.00          0.05       transfer   shipped
   O1015        C012 2026-09-05T14:20:00Z            Books         1       25.00          0.00           card      paid
   O1016        C001 2026-09-06T11:10:00Z          Fashion         2       55.00          0.15        ewallet      paid
   O1017        C013 2026-09-06T13:05:00Z      Electronics         1      799.00          0.10           card      paid
   O1018        C014 2026-09-06T17:45:00Z             Home         1      210.00          0.00       transfer   shipped
   O1019        C015 2026-09-07T08:35:00Z            Books         4        9.99          0.00           card      paid
   O1020        C016 2026-09-07T10:15:00Z          Fashion         1      110.00          0.25        ewallet      paid
   O1021        C017 2026-09-07T14:00:00Z      Electronics         2       35.00          0.00       transfer      paid
   O1022        C018 2026-09-08T09:30:00Z             Home         3       27.50          0.05           card   shipped
   O1023        C019 2026-09-08T12:10:00Z            Books         2       14.00          0.00        ewallet      paid
   O1024        C020 2026-09-08T18:25:00Z          Fashion         1      199.00          0.10           card      paid
   O1025        C001 2026-09-09T07:55:00Z      Electronics         1       59.00          0.00       transfer      paid
   O1026        C021 2026-09-09T11:35:00Z             Home         2       85.00          0.10           card   shipped
   O1027        C022 2026-09-09T15:20:00Z            Books         1       32.00          0.00        ewallet      paid
   O1028        C023 2026-09-10T08:05:00Z          Fashion         2       48.00          0.05           card      paid
   O1029        C024 2026-09-10T10:45:00Z      Electronics         1      349.00          0.10       transfer      paid
   O1030        C025 2026-09-10T16:30:00Z             Home         1      160.00          0.00           card   shipped
   O1031        C026 2026-09-11T09:00:00Z            Books         0       15.00          0.00           card      paid
   O1032        C027 2026-09-11T10:00:00Z          Fashion         1      -12.00          0.00        ewallet      paid
   O1033        C028 2026-09-11T11:00:00Z      Electronics         1       99.00          1.20           card      paid
   O1034        C029          not-a-date             Home         1       50.00          0.00       transfer      paid
   O1035        C030 2026-09-11T14:00:00Z            Books         2       20.00          0.00           card   pending
   O1036          NaN 2026-09-11T15:00:00Z          Fashion         1       30.00          0.00        ewallet      paid
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse>
```

11. buat modul validasi Pydantic



Welcome requirements.txt validate_orders.py

scripts > validate_orders.py > ...

```python
from pathlib import Path
from typing import Literal
from datetime import datetime

import pandas as pd
from pydantic import BaseModel, Field, ValidationError, StrictInt, field_validator


# Lokasi proyek berdasarkan folder kerja saat ini
BASE_DIR = Path(__file__).resolve().parent.parent
SOURCE_FILE = BASE_DIR / "data" / "source" / "orders.csv"
VALID_DIR = BASE_DIR / "data" / "valid"
REJECTED_DIR = BASE_DIR / "data" / "rejected"

VALID_FILE = VALID_DIR / "orders_valid.parquet"
REJECTED_FILE = REJECTED_DIR / "orders_rejected.parquet"
REJECTED_CSV = REJECTED_DIR / "orders_rejected.csv"


class OrderSchema(BaseModel):
    """Kontrak data untuk satu transaksi e-commerce."""

    order_id: str = Field(min_length=1)
    customer_id: str = Field(min_length=1)
    order_ts: datetime
    product_category: str = Field(min_length=1)
    quantity: StrictInt = Field(gt=0)
    unit_price: float = Field(ge=0)
    discount_pct: float = Field(ge=0, le=1)
    payment_method: str = Field(min_length=1)
    status: Literal["paid", "shipped", "cancelled", "refunded"]

    @field_validator(
        "order_id",
        "customer_id",
        "product_category",
        "payment_method",
        mode="before",
    )
    @classmethod
    def validate_non_empty_string(cls, value):
        if not isinstance(value, str):
            raise ValueError("Nilai wajib diisi dengan teks")
        value = value.strip()
        if not value:
            raise ValueError("Nilai tidak boleh kosong")
        return value

    @field_validator("order_ts", mode="before")
    @classmethod
    def validate_timestamp(cls, value):
        if not isinstance(value, str) or not value.strip():
            raise ValueError("order_ts wajib berupa timestamp yang valid")
        return value.strip()


def normalize_missing_values(row: dict) -> dict:
    """Mengubah NaN dari pandas menjadi None agar mudah divalidasi."""
    normalized = {}

    for key, value in row.items():
        if pd.isna(value):
            normalized[key] = None
        else:
            normalized[key] = value

    return normalized
```

```python
def format_validation_errors(error: ValidationError) -> str:
    """Menggabungkan seluruh alasan penolakan untuk satu baris."""
    messages = []

    for item in error.errors():
        field = ".".join(str(part) for part in item["loc"])
        messages.append(f"{field}: {item['msg']}")

    return "; ".join(messages)


def main():
    VALID_DIR.mkdir(parents=True, exist_ok=True)
    REJECTED_DIR.mkdir(parents=True, exist_ok=True)

    if not SOURCE_FILE.exists():
        raise FileNotFoundError(
            f"File sumber tidak ditemukan: {SOURCE_FILE}"
        )

    df = pd.read_csv(SOURCE_FILE)

    required_columns = [
        "order_id",
        "customer_id",
        "order_ts",
        "product_category",
        "quantity",
        "unit_price",
        "discount_pct",
        "payment_method",
```

```
    missing_columns = [
        column for column in required_columns if column not in df.columns
    ]

    if missing_columns:
        raise ValueError(
            f"Kolom wajib tidak ditemukan: {missing_columns}"
        )

    valid_rows = []
    rejected_rows = []

    for row_number, row in enumerate(
        df[required_columns].to_dict(orient="records"),
        start=2,  # Header CSV berada pada baris 1
    ):
        raw_row = normalize_missing_values(row)

        try:
            order = OrderSchema.model_validate(raw_row)

            clean_row = order.model_dump()
            valid_rows.append(clean_row)

        except ValidationError as error:
            rejected_row = raw_row.copy()
            rejected_row["source_row"] = row_number
            rejected_row["rejection_reason"] = format_validation_errors(error)
            rejected_rows.append(rejected_row)

    valid_df = pd.DataFrame(valid_rows, columns=required_columns)
    rejected_df = pd.DataFrame(
        rejected_rows,
        columns=required_columns + ["source_row", "rejection_reason"],
    )

    # Menyimpan timestamp yang valid dalam format datetime UTC.
    if not valid_df.empty:
        valid_df["order_ts"] = pd.to_datetime(
            valid_df["order_ts"], utc=True
        )

    valid_df.to_parquet(VALID_FILE, index=False)
    rejected_df.to_parquet(REJECTED_FILE, index=False)
    rejected_df.to_csv(REJECTED_CSV, index=False)

    print("=" * 55)
    print("HASIL VALIDASI DATA TRANSAKSI")
    print("=" * 55)
    print(f"Total baris sumber : {len(df)}")
    print(f"Data valid         : {len(valid_df)}")
    print(f"Data ditolak       : {len(rejected_df)}")
    print(f"Total diproses     : {len(valid_df) + len(rejected_df)}")
    print()
    print(f"Parquet valid      : {VALID_FILE.relative_to(BASE_DIR)}")
    print(f"Parquet rejected   : {REJECTED_FILE.relative_to(BASE_DIR)}")
    print(f"CSV rejected       : {REJECTED_CSV.relative_to(BASE_DIR)}")

    if not rejected_df.empty:
        print("\nDETAIL BARIS YANG DITOLAK:")
        print(
            rejected_df[
                ["order_id", "source_row", "rejection_reason"]
            ].to_string(index=False)
        )
    print("\nValidasi selesai.")


if __name__ == "__main__":
    main()
```

12. jalankan modul validasi

```
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> python scripts/validate_orders.py
==================================================
HASIL VALIDASI DATA TRANSAKSI
==================================================
Total baris sumber : 36
Data valid         : 30
Data ditolak       : 6
Total diproses     : 36

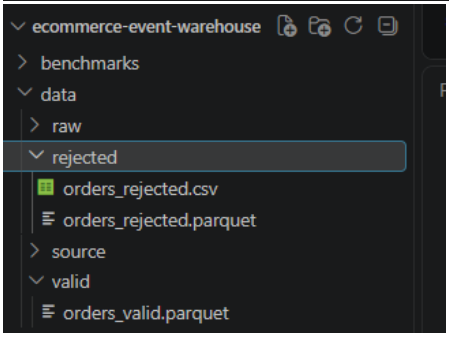
Parquet valid      : data\valid\orders_valid.parquet
Parquet rejected   : data\rejected\orders_rejected.parquet
CSV rejected       : data\rejected\orders_rejected.csv

DETAIL BARIS YANG DITOLAK:
order_id  source_row                                                    rejection_reason
   O1031          32                                 quantity: Input should be greater than 0
   O1032          33                       unit_price: Input should be greater than or equal to 0
   O1033          34                        discount_pct: Input should be less than or equal to 1
   O1034          35 order_ts: Input should be a valid datetime or date, invalid character in year
   O1035          36          status: Input should be 'paid', 'shipped', 'cancelled' or 'refunded'
   O1036          37                      customer_id: Value error, Nilai wajib diisi dengan teks

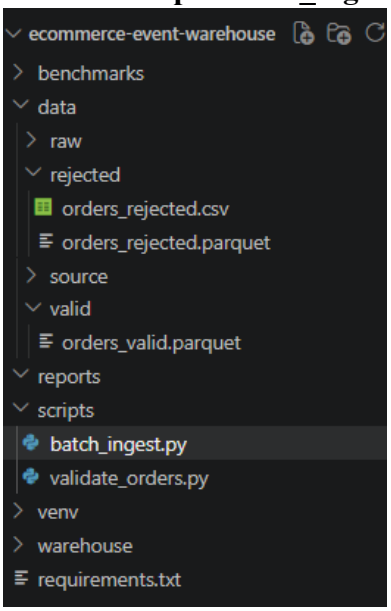
Validasi selesai.
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse>
```

13. periksa file hasilnya

```
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> python -c "import pandas as pd; print('VALID:', len(pd.read_parquet('data/valid/orders_valid.parquet'))); print('REJECTED:', len(pd.read_parquet('data/rejected/orders_rejected.parquet')))"
VALID: 30
REJECTED: 6
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse>
```



14. buat file scripts/batch_ingest.py



scripts > batch_ingest.py > ...

```python
from pathlib import Path
from datetime import datetime, timezone
import json
import time

import pandas as pd


BASE_DIR = Path(__file__).resolve().parent.parent

SOURCE_FILE = BASE_DIR / "data" / "source" / "orders.csv"
RAW_DIR = BASE_DIR / "data" / "raw"
RAW_FILE = RAW_DIR / "orders_raw.parquet"
WATERMARK_FILE = RAW_DIR / "watermark.json"

PAGE_SIZE = 10
MAX_RETRIES = 3


def read_source_page(source_df, page, page_size):
    """
    Simulasi endpoint API dengan pagination.
    Page dimulai dari 1.
    """
    if page < 1:
        raise ValueError("Nomor page harus >= 1")

    start = (page - 1) * page_size
    end = start + page_size

    return source_df.iloc[start:end].copy()


def fetch_page_with_retry(source_df, page, page_size):
    """
    Mengambil satu halaman dengan retry exponential backoff.

    Untuk demo, kegagalan sementara disimulasikan satu kali
    pada halaman kedua.
    """
    simulated_failure_done = False

    for attempt in range(MAX_RETRIES):
        try:
            # Simulasi satu kegagalan sementara agar retry terlihat.
            if page == 2 and not simulated_failure_done:
                simulated_failure_done = True
                raise ConnectionError(
                    "Simulasi kegagalan sementara saat mengambil page 2"
                )

            return read_source_page(source_df, page, page_size)

        except ConnectionError as error:
            if attempt == MAX_RETRIES - 1:
                raise

            wait_seconds = 2 ** attempt
            print(
                f"[RETRY] Page {page}, percobaan {attempt + 1}: "
                f"{error}. Menunggu {wait_seconds} detik."
            )
            time.sleep(wait_seconds)

    return pd.DataFrame(columns=source_df.columns)
```

```python
def load_watermark():
    """Membaca watermark sebelumnya jika tersedia."""
    if not WATERMARK_FILE.exists():
        return None

    with open(WATERMARK_FILE, "r", encoding="utf-8") as file:
        state = json.load(file)

    return state.get("watermark")


def save_watermark(value):
    """Menyimpan watermark terakhir yang berhasil diproses."""
    state = {
        "watermark": value,
        "updated_at": datetime.now(timezone.utc).isoformat(),
    }

    with open(WATERMARK_FILE, "w", encoding="utf-8") as file:
        json.dump(state, file, indent=2)


def main():
    RAW_DIR.mkdir(parents=True, exist_ok=True)

    if not SOURCE_FILE.exists():
        raise FileNotFoundError(f"Sumber tidak ditemukan: {SOURCE_FILE}")

    source_df = pd.read_csv(SOURCE_FILE)

    required_columns = {"order_id", "order_ts"}
    if not required_columns.issubset(source_df.columns):
        raise ValueError(
```

```
            f"Kolom wajib tidak ditemukan: {required_columns}"
        )

    # Baca watermark dan data raw sebelumnya.
    previous_watermark = load_watermark()

    if RAW_FILE.exists():
        existing_df = pd.read_parquet(RAW_FILE)
    else:
        existing_df = pd.DataFrame(columns=source_df.columns)

    # Ambil data per halaman, bukan membaca seluruh sumber sekaligus.
    fetched_pages = []
    page = 1

    while True:
        page_df = fetch_page_with_retry(
            source_df,
            page=page,
            page_size=PAGE_SIZE,
        )

        if page_df.empty:
            break

        fetched_pages.append(page_df)
        print(f"[INGEST] Page {page}: {len(page_df)} baris")

        if len(page_df) < PAGE_SIZE:
            break

        page += 1

    if fetched_pages:
        fetched_df = pd.concat(fetched_pages, ignore_index=True)
    else:
        fetched_df = pd.DataFrame(columns=source_df.columns)

    # Normalisasi timestamp untuk perbandingan watermark.
    fetched_df["_watermark_ts"] = pd.to_datetime(
        fetched_df["order_ts"],
        utc=True,
        errors="coerce",
    )

    if previous_watermark:
        watermark_ts = pd.to_datetime(previous_watermark, utc=True)
        incremental_df = fetched_df[
            fetched_df["_watermark_ts"] > watermark_ts
        ].copy()
    else:
        incremental_df = fetched_df.copy()

    # Hindari duplikasi order_id terhadap data yang sudah tersimpan.
    existing_ids = set(existing_df["order_id"].dropna().astype(str))
    incremental_df = incremental_df[
        ~incremental_df["order_id"].astype(str).isin(existing_ids)
    ].copy()

    # Jangan menyimpan kolom bantu ke output.
    incremental_df = incremental_df.drop(
        columns=["_watermark_ts"],
        errors="ignore",
    )

    if not incremental_df.empty:
        combined_df = pd.concat(
            [existing_df, incremental_df],
            ignore_index=True,
        )

        # Pertahankan satu baris per order_id.
        combined_df = combined_df.drop_duplicates(
            subset=["order_id"],
            keep="last",
        )

        combined_df.to_parquet(RAW_FILE, index=False)

        # Perbarui watermark berdasarkan timestamp valid tertinggi
        # dari data yang benar-benar masuk pada proses ini.
        new_timestamps = pd.to_datetime(
            incremental_df["order_ts"],
            utc=True,
            errors="coerce",
        ).dropna()

        if not new_timestamps.empty:
            latest_timestamp = new_timestamps.max().isoformat()
            save_watermark(latest_timestamp)

    print("\n" + "=" * 50)
    print("HASIL BATCH INGESTION")
    print("=" * 50)
    print(f"Baris sumber       : {len(source_df)}")
    print(f"Baris diambil      : {len(fetched_df)}")
    print(f"Watermark awal     : {previous_watermark}")
    print(f"Baris baru         : {len(incremental_df)}")

    if RAW_FILE.exists():
```

```
        final_df = pd.read_parquet(RAW_FILE)
        print(f"Total raw tersimpan: {len(final_df)}")
        print(f"Duplikat order_id  : {final_df['order_id'].duplicated().sum()}")
    else:
        print("Total raw tersimpan: 0")

    print(f"File raw           : {RAW_FILE.relative_to(BASE_DIR)}")
    print(f"File watermark     : {WATERMARK_FILE.relative_to(BASE_DIR)}")


if __name__ == "__main__":
    main()

```

15. jalankan dan uji idempotensi

```
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> python scripts/batch_ingest.py
[INGEST] Page 1: 10 baris
[RETRY] Page 2, percobaan 1: Simulasi kegagalan sementara saat mengambil page 2. Menunggu 1 detik.
[INGEST] Page 2: 10 baris
[INGEST] Page 3: 10 baris
[INGEST] Page 4: 6 baris
C:\Users\LEGION\Downloads\ecommerce-event-warehouse\scripts\batch_ingest.py:167: FutureWarning: The behavior of DataFrame concatenation with empty or all-NA entries
 deprecated. In a future version, this will no longer exclude empty or all-NA columns when determining the result dtypes. To retain the old behavior, exclude the rel
ant entries before the concat operation.
  combined_df = pd.concat(

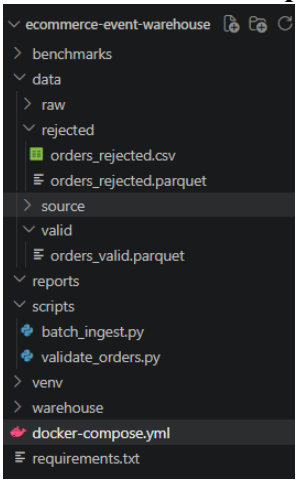
==================================================
HASIL BATCH INGESTION
==================================================
Baris sumber       : 36
Baris diambil      : 36
Watermark awal     : None
Baris baru         : 36
Total raw tersimpan: 36
Duplikat order_id  : 0
File raw           : data\raw\orders_raw.parquet
File watermark     : data\raw\watermark.json
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse>
```

16. lalu jalankan sekali lagi

```
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> python scripts/batch_ingest.py
[INGEST] Page 1: 10 baris
[RETRY] Page 2, percobaan 1: Simulasi kegagalan sementara saat mengambil page 2. Menunggu 1 detik.
[INGEST] Page 2: 10 baris
[INGEST] Page 3: 10 baris
[INGEST] Page 4: 6 baris

==================================================
HASIL BATCH INGESTION
==================================================
Baris sumber       : 36
Baris diambil      : 36
Watermark awal     : 2026-09-11T15:00:00+00:00
Baris baru         : 0
Total raw tersimpan: 36
Duplikat order_id  : 0
File raw           : data\raw\orders_raw.parquet
File watermark     : data\raw\watermark.json
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse>
```

17. buat file docker-compose.yml



docker-compose.yml

```yaml
services:
  kafka:
    image: apache/kafka:3.9.1
    container_name: ecommerce-kafka
    hostname: kafka
    ports:
      - "9092:9092"
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
      KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
      KAFKA_NUM_PARTITIONS: 3
    volumes:
      - kafka_data:/var/lib/kafka/data
    healthcheck:
      test:
        [
          "CMD-SHELL",
          "/opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --list >/dev/null 2>&1"
        ]
      interval: 10s
      timeout: 10s
      retries: 10
      start_period: 20s
    restart: unless-stopped

volumes:
  kafka_data:
```

18. jalankan kafka

```
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> docker compose up -d
 [+] Running 10/10
  ✓ kafka Pulled
    ✓ f7cdaeb78991 Pull complete
    ✓ a34edb4da622 Pull complete
    ✓ cda86626eeb3 Pull complete
    ✓ 34d370b15c91 Pull complete
    ✓ 304e58a73598 Pull complete
    ✓ f6cd406c8d97 Pull complete
    ✓ 6e6e21ba9841 Pull complete
    ✓ 74f6a226ed93 Pull complete
    ✓ e6744199aa66 Pull complete
 [+] Running 3/3
  ✓ Network ecommerce-event-warehouse_default      Created
  ✓ Volume "ecommerce-event-warehouse_kafka_data"  Created
  ✓ Container ecommerce-kafka                      Started
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> docker compose ps
NAME              IMAGE                COMMAND                  SERVICE   CREATED          STATUS                    PORTS
ecommerce-kafka   apache/kafka:3.9.1   "/__cacert_entrypoin…"   kafka     39 seconds ago   Up 39 seconds (healthy)   0.0.0.0:9092->9092/tcp
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse>
```

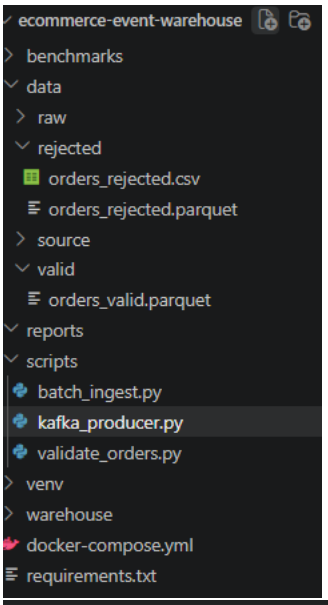
19. buat dan periksa topic orders

```
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> docker exec ecommerce-kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --create
--topic orders --partitions 3 --replication-factor 1
Created topic orders.
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse>
```

20. kemudian periksa

```
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> docker exec ecommerce-kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --describe
--topic orders
Topic: orders   TopicId: 1T5zjOntQ8afVuRJBDiLkA PartitionCount: 3       ReplicationFactor: 1    Configs:
        Topic: orders   Partition: 0    Leader: 1       Replicas: 1     Isr: 1  Elr:    LastKnownElr:
        Topic: orders   Partition: 1    Leader: 1       Replicas: 1     Isr: 1  Elr:    LastKnownElr:
        Topic: orders   Partition: 2    Leader: 1       Replicas: 1     Isr: 1  Elr:    LastKnownElr:
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse>
```

21. buat scripts/kafka_producer.py



scripts > kafka_producer.py > ...

```python
from pathlib import Path
import json
import time

import pandas as pd
from confluent_kafka import Producer


BASE_DIR = Path(__file__).resolve().parent.parent
SOURCE_FILE = BASE_DIR / "data" / "source" / "orders.csv"

BOOTSTRAP_SERVERS = "localhost:9092"
TOPIC = "orders"


def delivery_report(error, message):
    """Melaporkan hasil pengiriman setiap event."""
    if error is not None:
        print(f"[GAGAL] Event tidak terkirim: {error}")
    else:
        print(
            f"[TERKIRIM] order_id={message.key().decode('utf-8')} "
            f"partition={message.partition()} "
            f"offset={message.offset()}"
        )


def main():
    if not SOURCE_FILE.exists():
        raise FileNotFoundError(
            f"File sumber tidak ditemukan: {SOURCE_FILE}"
        )

    df = pd.read_csv(SOURCE_FILE)

    producer = Producer(
        {
            "bootstrap.servers": BOOTSTRAP_SERVERS,
            "client.id": "ecommerce-order-producer",
        }
    )

    success_count = 0

    print(f"Mengirim {len(df)} event ke topic '{TOPIC}'...")

    for row in df.to_dict(orient="records"):
        order_id = str(row.get("order_id", ""))

        # Ubah NaN dari pandas menjadi None agar JSON valid.
        clean_row = {
            key: None if pd.isna(value) else value
            for key, value in row.items()
        }
```

```
        payload = json.dumps(clean_row, ensure_ascii=False)

        # Key order_id menjaga event untuk order yang sama
        # diarahkan ke partisi yang sama.
        producer.produce(
            TOPIC,
            key=order_id.encode("utf-8"),
            value=payload.encode("utf-8"),
            callback=delivery_report,
        )

        success_count += 1

        # Layani callback dan batasi antrean producer.
        producer.poll(0)

    # Tunggu sampai seluruh pesan selesai dikirim.
    remaining = producer.flush(30)

    if remaining > 0:
        raise RuntimeError(
            f"{remaining} event belum selesai dikirim setelah timeout"
        )

    print("\n" + "=" * 45)
    print("PENGIRIMAN PRODUCER SELESAI")
    print("=" * 45)
    print(f"Event yang diantrekan : {success_count}")
    print(f"Event belum terkirim  : {remaining}")
    print(f"Topic                 : {TOPIC}")

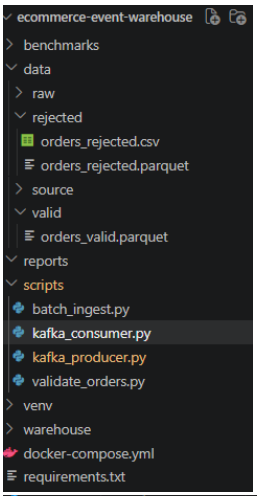

if __name__ == "__main__":
    main()
```

22. jalankan producer

```
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> python scripts/kafka_producer.py
Mengirim 36 event ke topic 'orders'...
[TERKIRIM] order_id=O1002 partition=1 offset=0
[TERKIRIM] order_id=O1005 partition=1 offset=1
[TERKIRIM] order_id=O1006 partition=1 offset=2
[TERKIRIM] order_id=O1011 partition=1 offset=3
[TERKIRIM] order_id=O1016 partition=1 offset=4
[TERKIRIM] order_id=O1022 partition=1 offset=5
[TERKIRIM] order_id=O1023 partition=1 offset=6
[TERKIRIM] order_id=O1030 partition=1 offset=7
[TERKIRIM] order_id=O1032 partition=1 offset=8
[TERKIRIM] order_id=O1033 partition=1 offset=9
[TERKIRIM] order_id=O1034 partition=1 offset=10
[TERKIRIM] order_id=O1035 partition=1 offset=11
[TERKIRIM] order_id=O1001 partition=2 offset=0
[TERKIRIM] order_id=O1008 partition=2 offset=1
[TERKIRIM] order_id=O1009 partition=2 offset=2
[TERKIRIM] order_id=O1012 partition=2 offset=3
[TERKIRIM] order_id=O1017 partition=2 offset=4
[TERKIRIM] order_id=O1018 partition=2 offset=5
[TERKIRIM] order_id=O1020 partition=2 offset=6
[TERKIRIM] order_id=O1024 partition=2 offset=7
[TERKIRIM] order_id=O1025 partition=2 offset=8
[TERKIRIM] order_id=O1031 partition=2 offset=9
[TERKIRIM] order_id=O1036 partition=2 offset=10
[TERKIRIM] order_id=O1003 partition=0 offset=0
[TERKIRIM] order_id=O1004 partition=0 offset=1
[TERKIRIM] order_id=O1007 partition=0 offset=2
[TERKIRIM] order_id=O1010 partition=0 offset=3
[TERKIRIM] order_id=O1013 partition=0 offset=4
[TERKIRIM] order_id=O1014 partition=0 offset=5
[TERKIRIM] order_id=O1015 partition=0 offset=6
[TERKIRIM] order_id=O1019 partition=0 offset=7
[TERKIRIM] order_id=O1021 partition=0 offset=8
[TERKIRIM] order_id=O1026 partition=0 offset=9
[TERKIRIM] order_id=O1027 partition=0 offset=10
[TERKIRIM] order_id=O1028 partition=0 offset=11
[TERKIRIM] order_id=O1029 partition=0 offset=12

=============================================
PENGIRIMAN PRODUCER SELESAI
=============================================
Event yang diantrekan : 36
Event belum terkirim  : 0
Topic                 : orders
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse>
```

23. buat scripts/kafka_consumer.py



scripts > kafka_consumer.py > ...

```python
from pathlib import Path
import json
import time

import pandas as pd
from confluent_kafka import Consumer, KafkaError
from pydantic import ValidationError

from validate_orders import OrderSchema, format_validation_errors


BASE_DIR = Path(__file__).resolve().parent.parent
VALID_DIR = BASE_DIR / "data" / "valid"
REJECTED_DIR = BASE_DIR / "data" / "rejected"

VALID_FILE = VALID_DIR / "orders_kafka_valid.parquet"
REJECTED_FILE = REJECTED_DIR / "orders_kafka_rejected.parquet"
REJECTED_CSV = REJECTED_DIR / "orders_kafka_rejected.csv"

BOOTSTRAP_SERVERS = "localhost:9092"
TOPIC = "orders"
CONSUMER_GROUP = "orders-validation-group"

# Berhenti jika tidak ada pesan baru selama 5 detik.
IDLE_TIMEOUT_SECONDS = 5


def load_existing(path):
    """Membaca hasil sebelumnya agar output tidak hilang saat rerun."""
    if path.exists():
        return pd.read_parquet(path)
    return pd.DataFrame()


def normalize_values(row):
    """Mengubah nilai NaN menjadi None."""
    return {
        key: None if pd.isna(value) else value
        for key, value in row.items()
    }


def main():
    VALID_DIR.mkdir(parents=True, exist_ok=True)
    REJECTED_DIR.mkdir(parents=True, exist_ok=True)

    consumer = Consumer({
        "bootstrap.servers": BOOTSTRAP_SERVERS,
        "group.id": CONSUMER_GROUP,
        "auto.offset.reset": "earliest",
        "enable.auto.commit": False,
    })

    consumer.subscribe([TOPIC])

    new_valid = []
    new_rejected = []
    received_count = 0
    last_message_time = time.monotonic()

    print(f"Mendengarkan topic '{TOPIC}'...")
    print(f"Consumer group: {CONSUMER_GROUP}")
    print("Menunggu pesan; akan berhenti setelah 5 detik idle.")
```

```python
    try:
        while True:
            message = consumer.poll(1.0)

            if message is None:
                if time.monotonic() - last_message_time >= IDLE_TIMEOUT_SECONDS:
                    break
                continue

            if message.error():
                if message.error().code() == KafkaError._PARTITION_EOF:
                    continue

                print(f"[KAFKA ERROR] {message.error()}")
                continue

            last_message_time = time.monotonic()
            received_count += 1

            partition = message.partition()
            offset = message.offset()

            try:
                raw = json.loads(message.value().decode("utf-8"))

                if not isinstance(raw, dict):
                    raise ValueError("Event JSON harus berupa objek")

                clean_raw = normalize_values(raw)
                order = OrderSchema.model_validate(clean_raw)
                record = order.model_dump()

                record["order_ts"] = pd.to_datetime(
                    record["order_ts"], utc=True
                )

                record["kafka_partition"] = partition
                record["kafka_offset"] = offset
                new_valid.append(record)

                print(
                    f"[VALID] {record['order_id']} "
                    f"(partition={partition}, offset={offset})"
                )

            except (ValidationError, ValueError, json.JSONDecodeError) as error:
                rejected = {
                    **(raw if "raw" in locals() and isinstance(raw, dict) else {}),
                    "kafka_partition": partition,
                    "kafka_offset": offset,
                    "rejection_reason": (
                        format_validation_errors(error)
                        if isinstance(error, ValidationError)
                        else str(error)
                    ),
                }
                new_rejected.append(rejected)

                print(
                    f"[REJECTED] order_id={rejected.get('order_id')} "
                    f"partition={partition} offset={offset}: "
                    f"{rejected['rejection_reason']}"
                )
```

```
            finally:
                # Mencegah variabel event sebelumnya terbawa ke event berikutnya
                if "raw" in locals():
                    del raw

        # Baca sink lama agar hasil sebelumnya tetap dipertahankan.
        old_valid = load_existing(VALID_FILE)
        old_rejected = load_existing(REJECTED_FILE)

        valid_columns = [
            "order_id", "customer_id", "order_ts", "product_category",
            "quantity", "unit_price", "discount_pct", "payment_method",
            "status", "kafka_partition", "kafka_offset",
        ]

        rejected_columns = [
            "order_id", "customer_id", "order_ts", "product_category",
            "quantity", "unit_price", "discount_pct", "payment_method",
            "status", "kafka_partition", "kafka_offset", "rejection_reason",
        ]

        new_valid_df = pd.DataFrame(new_valid)
        new_rejected_df = pd.DataFrame(new_rejected)

        if not old_valid.empty or not new_valid_df.empty:
            combined_valid = pd.concat(
                [old_valid, new_valid_df], ignore_index=True
            )
            combined_valid = combined_valid.drop_duplicates(
                subset=["order_id"], keep="first"
            )
            combined_valid.to_parquet(VALID_FILE, index=False)
        else:
            combined_valid = pd.DataFrame(columns=valid_columns)

        if not old_rejected.empty or not new_rejected_df.empty:
            combined_rejected = pd.concat(
                [old_rejected, new_rejected_df], ignore_index=True
            )

            # Satu hasil karantina per order_id untuk sink ini.
            # Untuk event tanpa order_id, gunakan lokasi event Kafka.
            if "order_id" not in combined_rejected.columns:
                combined_rejected["order_id"] = None

            combined_rejected["_dedupe_key"] = combined_rejected.apply(
                lambda row: (
                    f"order:{row['order_id']}"
                    if pd.notna(row.get("order_id"))
                    and str(row.get("order_id")).strip()
                    else (
                        f"kafka:{row.get('kafka_partition')}:"
                        f"{row.get('kafka_offset')}"
                    )
                ),
                axis=1,
            )

            combined_rejected = combined_rejected.drop_duplicates(
                subset=["_dedupe_key"], keep="first"
            ).drop(columns=["_dedupe_key"])

            combined_rejected.to_parquet(REJECTED_FILE, index=False)
            combined_rejected.to_csv(REJECTED_CSV, index=False)
        else:
            combined_rejected = pd.DataFrame(columns=rejected_columns)
```

```
        # Commit setelah hasil sink ditulis.
        # Jika terjadi crash sebelum commit, pesan dapat dibaca ulang.
        if received_count > 0:
            consumer.commit(asynchronous=False)

        print("\n" + "=" * 50)
        print("HASIL KAFKA CONSUMER")
        print("=" * 50)
        print(f"Pesan diterima sesi ini : {received_count}")
        print(f"Event valid baru        : {len(new_valid)}")
        print(f"Event rejected baru     : {len(new_rejected)}")
        print(f"Total valid di sink     : {len(combined_valid)}")
        print(f"Total rejected di sink  : {len(combined_rejected)}")
        print(f"Valid Parquet           : {VALID_FILE.relative_to(BASE_DIR)}")
        print(f"Rejected Parquet        : {REJECTED_FILE.relative_to(BASE_DIR)}")
        print(f"Rejected CSV            : {REJECTED_CSV.relative_to(BASE_DIR)}")

    finally:
        consumer.close()


if __name__ == "__main__":
    main()
```

24. jalankan consumer

```
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> python scripts/kafka_consumer.py
Mendengarkan topic 'orders'...
Consumer group: orders-validation-group
Menunggu pesan; akan berhenti setelah 5 detik idle.
[VALID] O1001 (partition=2, offset=0)
[VALID] O1008 (partition=2, offset=1)
[VALID] O1009 (partition=2, offset=2)
[VALID] O1012 (partition=2, offset=3)
[VALID] O1017 (partition=2, offset=4)
[VALID] O1018 (partition=2, offset=5)
[VALID] O1020 (partition=2, offset=6)
[VALID] O1024 (partition=2, offset=7)
[VALID] O1025 (partition=2, offset=8)
[REJECTED] order_id=O1031 partition=2 offset=9: quantity: Input should be greater than 0
[REJECTED] order_id=O1036 partition=2 offset=10: customer_id: Value error, Nilai wajib diisi dengan teks
[VALID] O1002 (partition=1, offset=0)
[VALID] O1005 (partition=1, offset=1)
[VALID] O1006 (partition=1, offset=2)
[VALID] O1011 (partition=1, offset=3)
[VALID] O1016 (partition=1, offset=4)
[VALID] O1022 (partition=1, offset=5)
[VALID] O1023 (partition=1, offset=6)
[VALID] O1030 (partition=1, offset=7)
[REJECTED] order_id=O1032 partition=1 offset=8: unit_price: Input should be greater than or equal to 0
[REJECTED] order_id=O1033 partition=1 offset=9: discount_pct: Input should be less than or equal to 1
[REJECTED] order_id=O1034 partition=1 offset=10: order_ts: Input should be a valid datetime or date, invalid character in year
[REJECTED] order_id=O1035 partition=1 offset=11: status: Input should be 'paid', 'shipped', 'cancelled' or 'refunded'
[VALID] O1003 (partition=0, offset=0)
[VALID] O1004 (partition=0, offset=1)
[VALID] O1007 (partition=0, offset=2)
[VALID] O1010 (partition=0, offset=3)
[VALID] O1013 (partition=0, offset=4)
[VALID] O1014 (partition=0, offset=5)
[VALID] O1015 (partition=0, offset=6)
[VALID] O1019 (partition=0, offset=7)
[VALID] O1021 (partition=0, offset=8)
[VALID] O1026 (partition=0, offset=9)
[VALID] O1027 (partition=0, offset=10)
[VALID] O1028 (partition=0, offset=11)
[VALID] O1029 (partition=0, offset=12)

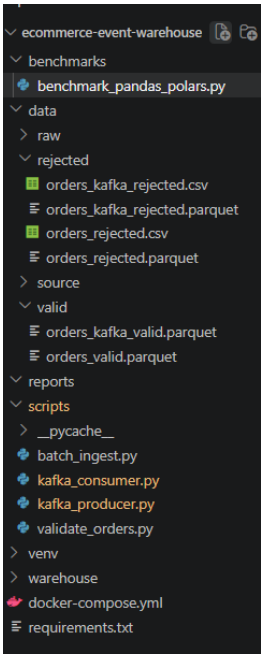
============================================
HASIL KAFKA CONSUMER
============================================
Pesan diterima sesi ini : 36
Event valid baru        : 30
Event rejected baru     : 6
Total valid di sink     : 30
Total rejected di sink  : 6
Valid Parquet           : data\valid\orders_kafka_valid.parquet
Rejected Parquet        : data\rejected\orders_kafka_rejected.parquet
Rejected CSV            : data\rejected\orders_kafka_rejected.csv
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse>
```

25. verifikasi hasilnya

```
  Rejected CSV            : data\rejected\orders_kafka_rejected.cpython -c "import pandas as pd; print('KAFKA VALID:', len(pd.read_parquet('data/valid/orders_kafka_valid.p
ad_parquet('data/rejected/orders_kafka_rejected.parquet')))"e>
  KAFKA VALID: 30
  KAFKA REJECTED: 6
  (venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse>
```

26. benchmark Pandas vs Polars

Buat file baru di VS Code:
`benchmarks/benchmark_pandas_polars.py`



```python
from pathlib import Path
from time import perf_counter
import gc
import pandas as pd
import polars as pl

BASE_DIR = Path(__file__).resolve().parent.parent
DATA_FILE = BASE_DIR / "data" / "raw" / "orders_raw.parquet"
RESULT_FILE = BASE_DIR / "benchmarks" / "benchmark_results.csv"

REPEATS = 10


def benchmark(name, function):
    # Pemanasan agar pengukuran awal tidak terlalu mendominasi.
    function()

    times = []

    for _ in range(REPEATS):
        gc.collect()
        start = perf_counter()
        result = function()
        elapsed = perf_counter() - start
        times.append(elapsed)
        del result

    return {
        "method": name,
        "repeats": REPEATS,
        "mean_seconds": sum(times) / len(times),
        "min_seconds": min(times),
        "max_seconds": max(times),
    }


def pandas_eager():
    df = pd.read_parquet(DATA_FILE)
    result = (
        df.groupby("product_category", as_index=False)
        .agg(
            total_quantity=("quantity", "sum"),
            total_orders=("order_id", "count"),
            average_price=("unit_price", "mean"),
        )
    )
    return result


def polars_eager():
    df = pl.read_parquet(DATA_FILE)
    result = (
        df.group_by("product_category")
        .agg(
            pl.col("quantity").sum().alias("total_quantity"),
            pl.col("order_id").count().alias("total_orders"),
            pl.col("unit_price").mean().alias("average_price"),
        )
    )
    return result


def polars_lazy():
    query = (
        pl.scan_parquet(DATA_FILE)
        .group_by("product_category")
        .agg(
            pl.col("quantity").sum().alias("total_quantity"),
            pl.col("order_id").count().alias("total_orders"),
            pl.col("unit_price").mean().alias("average_price"),
        )
    )
    return query.collect()


def main():
    if not DATA_FILE.exists():
        raise FileNotFoundError(f"Dataset tidak ditemukan: {DATA_FILE}")

    print(f"Dataset: {DATA_FILE}")
    print(f"Jumlah baris: {len(pd.read_parquet(DATA_FILE))}")
    print(f"Pengulangan per metode: {REPEATS}")
    print("\nMenjalankan benchmark...\n")

    results = [
        benchmark("Pandas Eager", pandas_eager),
        benchmark("Polars Eager", polars_eager),
        benchmark("Polars Lazy", polars_lazy),
    ]

    results_df = pd.DataFrame(results)
    results_df.to_csv(RESULT_FILE, index=False)

    print(results_df.to_string(index=False))
    print(f"\nHasil tersimpan di: {RESULT_FILE}")


if __name__ == "__main__":
    main()
```

27. jalankan benchmark

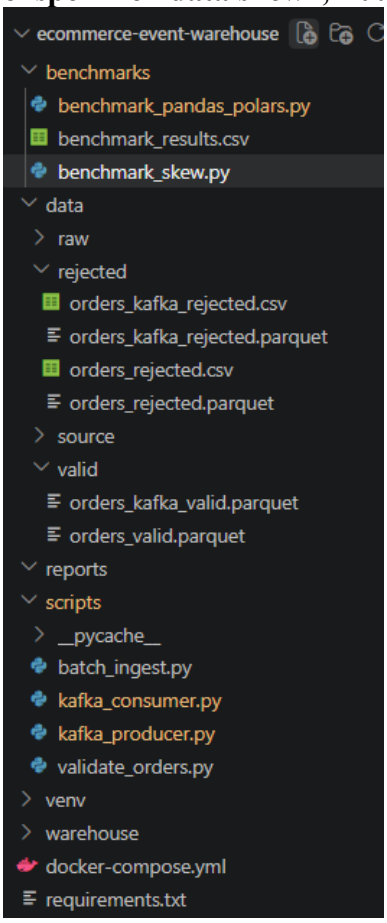
```
KAFKA REJECTED: 0
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> python benchmarks/benchmark_pandas_polars.py
Dataset: C:\Users\LEGION\Downloads\ecommerce-event-warehouse\data\raw\orders_raw.parquet
Jumlah baris: 36
Pengulangan per metode: 10

Menjalankan benchmark...

      method  repeats  mean_seconds  min_seconds  max_seconds
Pandas Eager       10      0.008101     0.006190     0.011381
Polars Eager       10      0.001927     0.001700     0.002150
 Polars Lazy       10      0.001804     0.001502     0.002038

Hasil tersimpan di: C:\Users\LEGION\Downloads\ecommerce-event-warehouse\benchmarks\benchmark_results.csv
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse>
```

28. eksperimen data skew , Buat file eksperimen :



benchmarks > benchmark_skew.py > generate_datasets

```python
from pathlib import Path
from time import perf_counter
import gc

import numpy as np
import pandas as pd
import polars as pl

BASE_DIR = Path(__file__).resolve().parent.parent
DATA_DIR = BASE_DIR / "data" / "source"
RESULT_FILE = BASE_DIR / "benchmarks" / "skew_results.csv"

N_ROWS = 200_000
N_CATEGORIES = 20
REPEATS = 5
SEED = 42

CATEGORIES = [f"category_{i:02d}" for i in range(N_CATEGORIES)]


def generate_datasets():
    DATA_DIR.mkdir(parents=True, exist_ok=True)
    rng = np.random.default_rng(SEED)

    # Dataset balanced: peluang kategori relatif merata.
    balanced_categories = rng.choice(CATEGORIES, size=N_ROWS)

    # Dataset skewed: 95% kategori utama, sisanya tersebar
    # pada 19 kategori lainnya.
    skewed_categories = np.empty(N_ROWS, dtype=object)
    main_mask = rng.random(N_ROWS) < 0.95
    skewed_categories[main_mask] = CATEGORIES[0]
    skewed_categories[~main_mask] = rng.choice(
        CATEGORIES[1:],
        size=(~main_mask).sum(),
    )

    for name, categories in [
        ("balanced", balanced_categories),
        ("skewed", skewed_categories),
    ]:
        df = pd.DataFrame({
            "order_id": [f"{name}_{i:07d}" for i in range(N_ROWS)],
            "product_category": categories,
            "quantity": rng.integers(1, 6, size=N_ROWS),
            "unit_price": rng.uniform(10_000, 500_000, size=N_ROWS),
        })

        output = DATA_DIR / f"orders_{name}.parquet"
        df.to_parquet(output, index=False)

        proportions = df["product_category"].value_counts(normalize=True)
        print(f"\nDataset {name}:")
        print(f"Jumlah baris       : {len(df):,}")
        print(f"Jumlah kategori    : {df['product_category'].nunique()}")
        print(f"Proporsi kategori terbesar: {proportions.max():.2%}")
        print(f"File: {output}")


def pandas_eager(path):
    df = pd.read_parquet(path)
    return (
        df.groupby("product_category", as_index=False)
        .agg(
            total_quantity=("quantity", "sum"),
            total_orders=("order_id", "count"),
            average_price=("unit_price", "mean"),
        )
    )


def polars_eager(path):
    df = pl.read_parquet(path)
    return (
        df.group_by("product_category")
        .agg(
            pl.col("quantity").sum().alias("total_quantity"),
            pl.col("order_id").count().alias("total_orders"),
            pl.col("unit_price").mean().alias("average_price"),
        )
    )


def polars_lazy(path):
    return (
        pl.scan_parquet(path)
        .group_by("product_category")
        .agg(
            pl.col("quantity").sum().alias("total_quantity"),
            pl.col("order_id").count().alias("total_orders"),
            pl.col("unit_price").mean().alias("average_price"),
        )
        .collect()
    )


def measure(method_name, dataset_name, function):
    # Warm-up tidak dimasukkan ke rata-rata.
    function()
```

```
    times = []
    for _ in range(REPEATS):
        gc.collect()
        start = perf_counter()
        result = function()
        times.append(perf_counter() - start)
        del result

    return {
        "dataset": dataset_name,
        "method": method_name,
        "rows": N_ROWS,
        "repeats": REPEATS,
        "mean_seconds": sum(times) / len(times),
        "min_seconds": min(times),
        "max_seconds": max(times),
    }


def main():
    generate_datasets()
    results = []

    methods = [
        ("Pandas Eager", pandas_eager),
        ("Polars Eager", polars_eager),
        ("Polars Lazy", polars_lazy),
    ]

    for dataset_name in ["balanced", "skewed"]:
        path = DATA_DIR / f"orders_{dataset_name}.parquet"

        for method_name, function in methods:
            print(f"Mengukur {method_name} pada {dataset_name}...")
            results.append(
                measure(
                    method_name,
                    dataset_name,
                    lambda fn=function, p=path: fn(p),
                )
            )

    result_df = pd.DataFrame(results)
    result_df.to_csv(RESULT_FILE, index=False)

    print("\n=== HASIL EKSPERIMEN SKEW ===")
    print(
        result_df[
            ["dataset", "method", "mean_seconds",
             "min_seconds", "max_seconds"]
        ].to_string(index=False)
    )
    print(f"\nHasil tersimpan di: {RESULT_FILE}")
```

29. jalankan eksperimen

```
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> python benchmarks/benchmark_skew.py

Dataset balanced:
Jumlah baris       : 200,000
Jumlah kategori    : 20
Proporsi kategori terbesar: 5.08%
File: C:\Users\LEGION\Downloads\ecommerce-event-warehouse\data\source\orders_balanced.parquet

Dataset skewed:
Jumlah baris       : 200,000
Jumlah kategori    : 20
Proporsi kategori terbesar: 95.04%
File: C:\Users\LEGION\Downloads\ecommerce-event-warehouse\data\source\orders_skewed.parquet
Mengukur Pandas Eager pada balanced...
Mengukur Polars Eager pada balanced...
Mengukur Polars Lazy pada balanced...
Mengukur Pandas Eager pada skewed...
Mengukur Polars Eager pada skewed...
Mengukur Polars Lazy pada skewed...

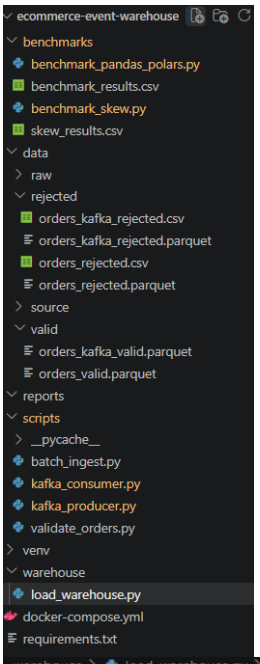
=== HASIL EKSPERIMEN SKEW ===
 dataset       method  mean_seconds  min_seconds  max_seconds
balanced Pandas Eager      0.079601     0.069964     0.090268
balanced Polars Eager      0.011365     0.010041     0.013402
balanced  Polars Lazy      0.010239     0.009376     0.011027
  skewed Pandas Eager      0.064511     0.061436     0.068689
  skewed Polars Eager      0.012172     0.011471     0.013386
  skewed  Polars Lazy      0.012443     0.011415     0.013487

Hasil tersimpan di: C:\Users\LEGION\Downloads\ecommerce-event-warehouse\benchmarks\skew_results.csv
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse>
```

30. Gambaran pipeline yang akan dibangun



31. Masukkan data ke DuckDB. Buat file baru bernama warehouse/load_warehouse.py





```
            f"File data valid tidak ditemukan: {PARQUET_FILE}"
        )

    con = duckdb.connect(str(DB_FILE))

    try:
        parquet_path = PARQUET_FILE.as_posix().replace("'", "''")

        con.execute(f"""
            CREATE OR REPLACE TABLE raw_orders AS
            SELECT *
            FROM read_parquet('{parquet_path}')
        """)

        result = con.execute("""
            SELECT
                COUNT(*) AS total_rows,
                COUNT(DISTINCT order_id) AS unique_orders,
                MIN(order_ts) AS first_order_ts,
                MAX(order_ts) AS last_order_ts
            FROM raw_orders
        """).fetchone()

        print("=" * 45)
        print("HASIL LOAD DATA KE DUCKDB")
        print("=" * 45)
        print(f"Total baris       : {result[0]}")
        print(f"Order unik        : {result[1]}")
        print(f"Transaksi pertama : {result[2]}")
        print(f"Transaksi terakhir: {result[3]}")
        print(f"Database          : {DB_FILE}")

        print("\nPreview data:")
        print(
            con.execute("""
                SELECT *
                FROM raw_orders
                ORDER BY order_id
                LIMIT 5
            """).df().to_string(index=False)
        )

    finally:
        con.close()

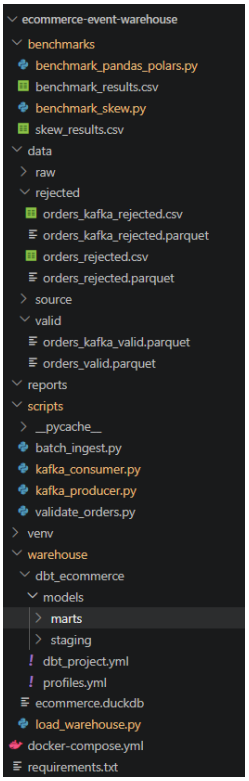

if __name__ == "__main__":
    main()
```

32. Jalankan dari terminal VS Code pada direktori utama proyek:

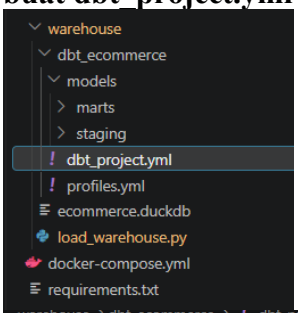
```
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> python warehouse/load_warehouse.py
=============================================
HASIL LOAD DATA KE DUCKDB
=============================================
Total baris       : 30
Order unik        : 30
Transaksi pertama : 2026-09-01 16:15:00+07:00
Transaksi terakhir: 2026-09-10 23:30:00+07:00
Database          : C:\Users\LEGION\Downloads\ecommerce-event-warehouse\warehouse\ecommerce.duckdb

Preview data:
order_id customer_id                order_ts product_category  quantity  unit_price  discount_pct payment_method    status
   O1001        C001 2026-09-01 16:15:00+07:00      Electronics         1      250.00          0.05           card      paid
   O1002        C002 2026-09-01 17:10:00+07:00            Home         2       45.50          0.00       transfer   shipped
   O1003        C003 2026-09-01 18:05:00+07:00           Books         3       12.99          0.10        ewallet      paid
   O1004        C001 2026-09-02 15:45:00+07:00      Electronics         1       89.90          0.15           card      paid
   O1005        C004 2026-09-02 19:00:00+07:00         Fashion         2       39.00          0.20        ewallet cancelled
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse>
```

33. konfigurasi dbt, buat struktur folder dbt



34. buat dbt_project.yml



warehouse › dbt_ecommerce › ! dbt_project.yml

```yaml

name: ecommerce_warehouse
version: "1.0.0"
config-version: 2

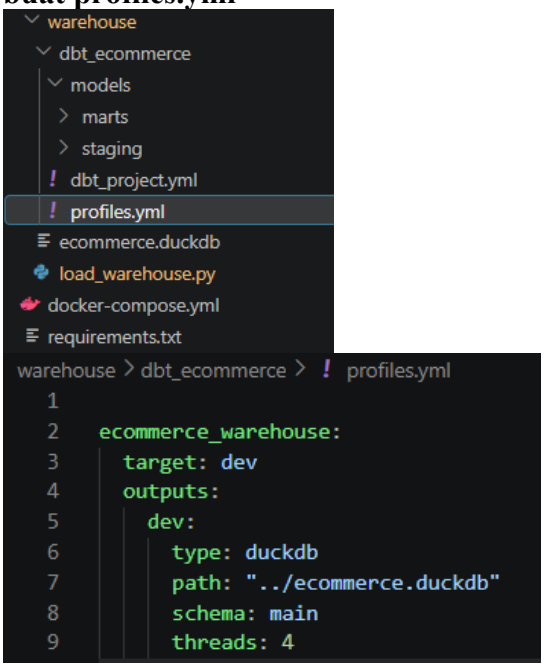
profile: ecommerce_warehouse

model-paths: ["models"]
analysis-paths: ["analyses"]
test-paths: ["tests"]
seed-paths: ["seeds"]
macro-paths: ["macros"]
snapshot-paths: ["snapshots"]

target-path: "target"
clean-targets:
  - "target"
  - "dbt_packages"

models:
  ecommerce_warehouse:
    staging:
      +materialized: view
    marts:
      +materialized: table
```

35. buat profiles.yml



36. uji koneksi dbt

```
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> cd warehouse/dbt_ecommerce
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse\warehouse\dbt_ecommerce> dbt --version
Core:
  - installed: 1.12.6
  - latest:    1.12.6 - Up to date!

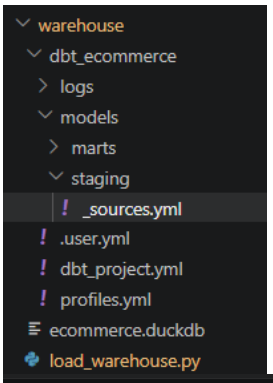
Plugins:
  - duckdb: 1.11.0 - Up to date!


06:44:08  dbt v2 is available, and is the default for new installations. This is dbt Core v1.
          New project - install v2: https://docs.getdbt.com/docs/local/install-dbt?utm_source=dbt-cli
          Existing v1 project - upgrade guide: https://docs.getdbt.com/docs/dbt-versions/dbt-upgrade/upgrading-to-v2?utm_source=dbt-cli
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse\warehouse\dbt_ecommerce> dbt debug --profiles-dir .
06:44:29  Running with dbt=1.12.6
06:44:29  dbt version: 1.12.6
06:44:29  python version: 3.10.0
06:44:29  python path: C:\Users\LEGION\Downloads\ecommerce-event-warehouse\venv\Scripts\python.exe
06:44:29  os info: Windows-10-10.0.19045-SP0
06:44:29  Using profiles dir at .
06:44:29  Using profiles.yml file at .\profiles.yml
06:44:29  Using dbt_project.yml file at C:\Users\LEGION\Downloads\ecommerce-event-warehouse\warehouse\dbt_ecommerce\dbt_project.yml
06:44:29  adapter type: duckdb
06:44:29  adapter version: 1.11.0
06:44:29  Configuration:
06:44:29    profiles.yml file [OK found and valid]
06:44:29    dbt_project.yml file [OK found and valid]
06:44:29  Required dependencies:
06:44:29   - git [OK found]

06:44:29  Connection:
06:44:29    database: ecommerce
06:44:29    schema: main
06:44:29    path: ../ecommerce.duckdb
06:44:29    config_options: None
06:44:29    extensions: None
06:44:29    settings: {}
06:44:29    external_root: .
06:44:29    use_credential_provider: None
06:44:29    attach: None
06:44:29    filesystems: None
06:44:29    remote: None
06:44:29    plugins: None
06:44:29    disable_transactions: False
06:44:29  Registered adapter: duckdb=1.11.0
06:44:29    Connection test: [OK connection ok]

06:44:29  All checks passed!
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse\warehouse\dbt_ecommerce>
```

37. buat source dbt. Di dalam models/staging, buat file _sources.yml

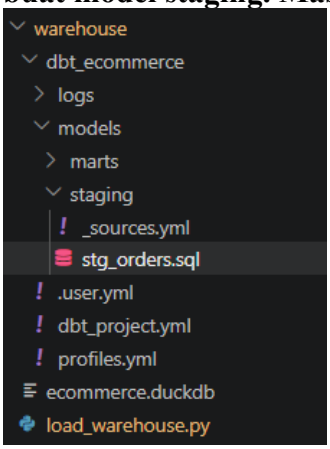


warehouse > dbt_ecommerce > models > staging > ! _sources.yml

```yaml
version: 2

sources:
  - name: ecommerce_source
    description: "Data transaksi yang sudah lolos validasi Pydantic."
    schema: main
    tables:
      - name: raw_orders
        description: "Tabel sumber transaksi dari file Parquet."
        columns:
          - name: order_id
            description: "ID unik setiap order."
          - name: customer_id
            description: "ID pelanggan."
          - name: order_ts
            description: "Waktu transaksi."
          - name: product_category
            description: "Kategori produk."
          - name: quantity
            description: "Jumlah produk yang dibeli."
          - name: unit_price
            description: "Harga satuan produk."
          - name: discount_pct
            description: "Persentase diskon dalam rentang 0 sampai 1."
          - name: payment_method
            description: "Metode pembayaran."
          - name: status
            description: "Status transaksi."
```

38. buat model staging. Masih di folder models/staging, buat file stg_orders.sql



warehouse > dbt_ecommerce > models > staging > stg_orders.sql

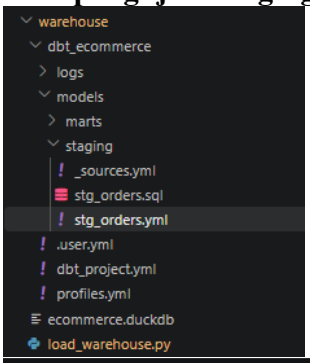
```sql

WITH source_data AS (
    SELECT *
    FROM {{ source('ecommerce_source', 'raw_orders') }}
),

cleaned AS (
    SELECT
        TRIM(order_id) AS order_id,
        TRIM(customer_id) AS customer_id,
        CAST(order_ts AS TIMESTAMPTZ) AS order_ts,
        CAST(order_ts AS DATE) AS order_date,
        TRIM(product_category) AS product_category,
        CAST(quantity AS INTEGER) AS quantity,
        CAST(unit_price AS DECIMAL(12, 2)) AS unit_price,
        CAST(discount_pct AS DECIMAL(5, 4)) AS discount_pct,
        TRIM(payment_method) AS payment_method,
        LOWER(TRIM(status)) AS status
    FROM source_data
)

SELECT
    *,
    CAST(quantity * unit_price AS DECIMAL(14, 2)) AS gross_amount,
    CAST(
        quantity * unit_price * discount_pct
        AS DECIMAL(14, 2)
    ) AS discount_amount,
    CAST(
        quantity * unit_price * (1 - discount_pct)
        AS DECIMAL(14, 2)
    ) AS net_amount
FROM cleaned
```

39. buat pengujian staging



warehouse > dbt_ecommerce > models > staging > stg_orders.yml

```yaml
version: 2

models:
  - name: stg_orders
    description: "Model staging transaksi yang sudah distandarisasi."
    columns:
      - name: order_id
        description: "ID order yang unik."
        data_tests:
          - not_null
          - unique

      - name: customer_id
        data_tests:
          - not_null

      - name: order_ts
        data_tests:
          - not_null

      - name: quantity
        data_tests:
          - not_null

      - name: unit_price
        data_tests:
          - not_null

      - name: discount_pct
        data_tests:
          - not_null

      - name: status
        data_tests:
          - not_null
```

40. jalankan model staging

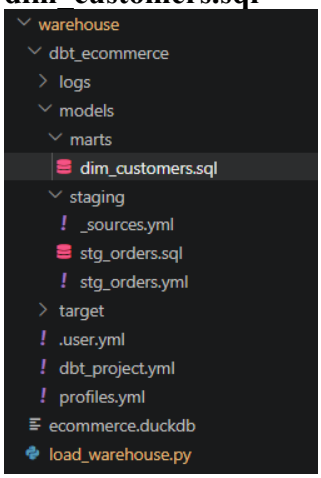
```
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse\warehouse\dbt_ecommerce> dbt run --select stg_orders --profiles-dir .
06:54:40  Running with dbt=1.12.6
06:54:41  Registered adapter: duckdb=1.11.0
06:54:41  Unable to do partial parsing because saved manifest not found. Starting full parse.
06:54:44  [WARNING]: Configuration paths exist in your dbt_project.yml file which do not apply to any resources.
There are 1 unused configuration paths:
- models.ecommerce_warehouse.marts
06:54:44  Found 1 model, 8 data tests, 1 source, 500 macros
06:54:44
06:54:44  Concurrency: 4 threads (target='dev')
06:54:44
06:54:44  1 of 1 START sql view model main.stg_orders ..................................... [RUN]
06:54:44  1 of 1 OK created sql view model main.stg_orders ................................ [OK in 0.09s]
06:54:44
06:54:44  Finished running 1 view model in 0 hours 0 minutes and 0.25 seconds (0.25s).
06:54:44
06:54:44  Completed successfully
06:54:44
06:54:44  Done. PASS=1 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=1
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse\warehouse\dbt_ecommerce>
```

41. lalu lanjutkan dengan ini

```
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse\warehouse\dbt_ecommerce> dbt test --select stg_orders --profiles-dir .
06:55:26  Running with dbt=1.12.6
06:55:26  Registered adapter: duckdb=1.11.0
06:55:26  [WARNING]: Configuration paths exist in your dbt_project.yml file which do not apply to any resources.
There are 1 unused configuration paths:
- models.ecommerce_warehouse.marts
06:55:27  Found 1 model, 8 data tests, 1 source, 500 macros
06:55:27
06:55:27  Concurrency: 4 threads (target='dev')
06:55:27
06:55:27  1 of 8 START test not_null_stg_orders_customer_id ............................... [RUN]
06:55:27  2 of 8 START test not_null_stg_orders_discount_pct .............................. [RUN]
06:55:27  3 of 8 START test not_null_stg_orders_order_id .................................. [RUN]
06:55:27  4 of 8 START test not_null_stg_orders_order_ts .................................. [RUN]
06:55:27  1 of 8 PASS not_null_stg_orders_customer_id ..................................... [PASS in 0.10s]
06:55:27  3 of 8 PASS not_null_stg_orders_order_id ........................................ [PASS in 0.08s]
06:55:27  4 of 8 PASS not_null_stg_orders_order_ts ........................................ [PASS in 0.08s]
06:55:27  5 of 8 START test not_null_stg_orders_quantity .................................. [RUN]
06:55:27  2 of 8 PASS not_null_stg_orders_discount_pct .................................... [PASS in 0.11s]
06:55:27  6 of 8 START test not_null_stg_orders_status .................................... [RUN]
06:55:27  7 of 8 START test not_null_stg_orders_unit_price ................................ [RUN]
06:55:27  8 of 8 START test unique_stg_orders_order_id .................................... [RUN]
06:55:27  5 of 8 PASS not_null_stg_orders_quantity ........................................ [PASS in 0.08s]
06:55:27  6 of 8 PASS not_null_stg_orders_status .......................................... [PASS in 0.07s]
06:55:27  7 of 8 PASS not_null_stg_orders_unit_price ...................................... [PASS in 0.07s]
06:55:27  8 of 8 PASS unique_stg_orders_order_id .......................................... [PASS in 0.06s]
06:55:27
06:55:27  Finished running 8 data tests in 0 hours 0 minutes and 0.34 seconds (0.34s).
06:55:27
06:55:27  Completed successfully
06:55:27
06:55:27  Done. PASS=8 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=8
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse\warehouse\dbt_ecommerce>
```

42. buat model dimensi. Di dalam models/marts/, buat file-file berikut:

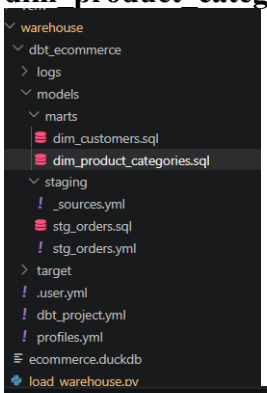
a. dim_customers.sql



warehouse > dbt_ecommerce > models > marts > dim_customers.sql

```sql
SELECT
    customer_id,
    MIN(order_ts) AS first_order_ts,
    MAX(order_ts) AS last_order_ts,
    COUNT(DISTINCT order_id) AS total_orders
FROM {{ ref('stg_orders') }}
GROUP BY customer_id
```

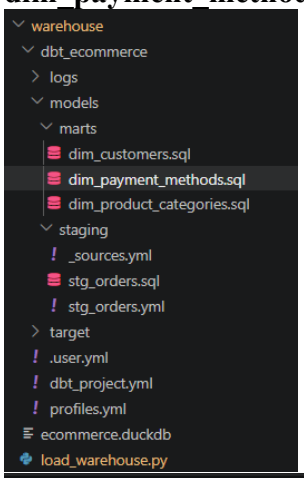
b. dim_product_categories.sql



warehouse > dbt_ecommerce > models > marts > dim_product_categories.sql

```sql
SELECT
    product_category,
    COUNT(DISTINCT order_id) AS total_orders
FROM {{ ref('stg_orders') }}
GROUP BY product_category
```

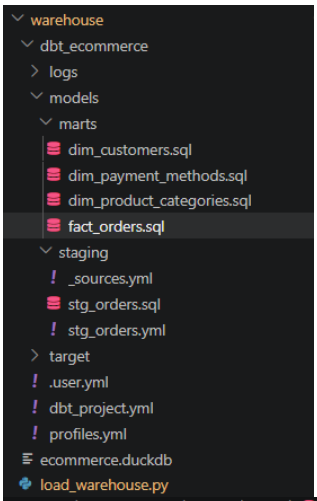
c. dim_payment_methods.sql



warehouse > dbt_ecommerce > models > marts > dim_payment_methods.sql

```sql
SELECT
    payment_method,
    COUNT(DISTINCT order_id) AS total_orders
FROM {{ ref('stg_orders') }}
GROUP BY payment_method
```

43. buat tabel fakta transaksi. Buat file models/marts/fact_orders.sql.



warehouse > dbt_ecommerce > models > marts > fact_orders.sql

```sql

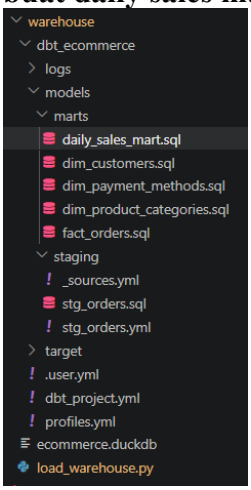
SELECT
    o.order_id,
    o.customer_id,
    o.product_category,
    o.payment_method,
    o.order_ts,
    o.order_date,
    o.quantity,
    o.unit_price,
    o.discount_pct,
    o.gross_amount,
    o.discount_amount,
    o.net_amount,
    o.status
FROM {{ ref('stg_orders') }} AS o

INNER JOIN {{ ref('dim_customers') }} AS c
    ON o.customer_id = c.customer_id

INNER JOIN {{ ref('dim_product_categories') }} AS p
    ON o.product_category = p.product_category

INNER JOIN {{ ref('dim_payment_methods') }} AS m
    ON o.payment_method = m.payment_method
```

44. buat daily sales mart

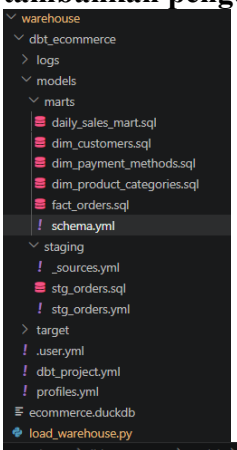


warehouse > dbt_ecommerce > models > marts > daily_sales_mart.sql

```sql

SELECT
    order_date,
    COUNT(DISTINCT order_id) AS total_orders,
    COUNT(DISTINCT customer_id) AS total_customers,
    SUM(quantity) AS total_quantity,
    ROUND(SUM(gross_amount), 2) AS total_gross_sales,
    ROUND(SUM(discount_amount), 2) AS total_discount,
    ROUND(SUM(net_amount), 2) AS total_net_sales,
    COUNT(*) FILTER (WHERE status = 'paid') AS paid_orders,
    COUNT(*) FILTER (WHERE status = 'shipped') AS shipped_orders,
    COUNT(*) FILTER (WHERE status = 'cancelled') AS cancelled_orders,
    COUNT(*) FILTER (WHERE status = 'refunded') AS refunded_orders
FROM {{ ref('fact_orders') }}
GROUP BY order_date
```

45. tambahkan pengujian kualitas data. Buat file models/marts/schema.yml.



warehouse > dbt_ecommerce > models > marts > schema.yml

```yaml
version: 2

models:
  - name: dim_customers
    description: "Dimensi pelanggan."
    columns:
      - name: customer_id
        data_tests:
          - unique
          - not_null

  - name: dim_product_categories
    description: "Dimensi kategori produk."
    columns:
      - name: product_category
        data_tests:
          - unique
          - not_null

  - name: dim_payment_methods
    description: "Dimensi metode pembayaran."
    columns:
      - name: payment_method
        data_tests:
          - unique
          - not_null

  - name: fact_orders
    description: "Fakta transaksi pada grain satu baris per order."
    columns:
      - name: order_id
        data_tests:
          - unique
          - not_null

      - name: customer_id
        data_tests:
          - not_null
          - relationships:
              arguments:
                to: ref('dim_customers')
                field: customer_id

      - name: product_category
        data_tests:
```

```yaml
      - not_null
      - relationships:
          arguments:
            to: ref('dim_product_categories')
            field: product_category

    - name: payment_method
      data_tests:
        - not_null
        - relationships:
            arguments:
              to: ref('dim_payment_methods')
              field: payment_method

  - name: daily_sales_mart
    description: "Agregasi penjualan harian."
    columns:
      - name: order_date
        data_tests:
          - unique
          - not_null
```

46. jalankan seluruh model. bangun seluruh model

```
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse\warehouse\dbt_ecommerce> dbt run --profiles-dir .
07:20:34  Running with dbt=1.12.6
07:20:34  Registered adapter: duckdb=1.11.0
07:20:36  Found 6 models, 24 data tests, 1 source, 500 macros
07:20:36
07:20:36  Concurrency: 4 threads (target='dev')
07:20:36
07:20:36  1 of 6 START sql view model main.stg_orders ..................................... [RUN]
07:20:36  1 of 6 OK created sql view model main.stg_orders ................................ [OK in 0.09s]
07:20:36  2 of 6 START sql table model main.dim_customers ................................. [RUN]
07:20:36  3 of 6 START sql table model main.dim_payment_methods .......................... [RUN]
07:20:36  4 of 6 START sql table model main.dim_product_categories ....................... [RUN]
07:20:36  2 of 6 OK created sql table model main.dim_customers ............................ [OK in 0.15s]
07:20:36  3 of 6 OK created sql table model main.dim_payment_methods ..................... [OK in 0.15s]
07:20:36  4 of 6 OK created sql table model main.dim_product_categories .................. [OK in 0.15s]
07:20:36  5 of 6 START sql table model main.fact_orders ................................... [RUN]
07:20:36  5 of 6 OK created sql table model main.fact_orders .............................. [OK in 0.03s]
07:20:36  6 of 6 START sql table model main.daily_sales_mart .............................. [RUN]
07:20:36  6 of 6 OK created sql table model main.daily_sales_mart ......................... [OK in 0.04s]
07:20:36
07:20:36  Finished running 5 table models, 1 view model in 0 hours 0 minutes and 0.54 seconds (0.54s).
07:20:36
07:20:36  Completed successfully
07:20:36
07:20:36  Done. PASS=6 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=6
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse\warehouse\dbt_ecommerce>
```

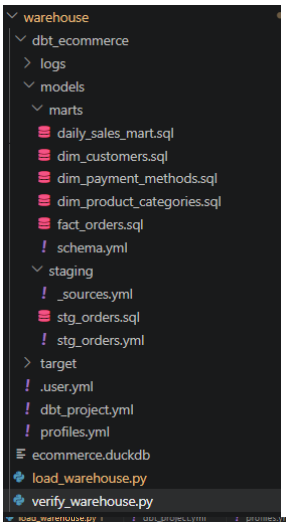
47. jalankan semua tes

```
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse\warehouse\dbt_ecommerce> dbt test --profiles-dir .
07:21:26  Running with dbt=1.12.6
07:21:26  Registered adapter: duckdb=1.11.0
07:21:27  Found 6 models, 24 data tests, 1 source, 500 macros
07:21:27
07:21:27  Concurrency: 4 threads (target='dev')
07:21:27
07:21:27  1 of 24 START test not_null_daily_sales_mart_order_date ........................ [RUN]
07:21:27  2 of 24 START test not_null_dim_customers_customer_id .......................... [RUN]
07:21:27  3 of 24 START test not_null_dim_payment_methods_payment_method ................. [RUN]
07:21:27  4 of 24 START test not_null_dim_product_categories_product_category ............ [RUN]
07:21:27  2 of 24 PASS not_null_dim_customers_customer_id ................................ [PASS in 0.09s]
07:21:27  5 of 24 START test not_null_fact_orders_customer_id ............................ [RUN]
07:21:27  3 of 24 PASS not_null_dim_payment_methods_payment_method ....................... [PASS in 0.09s]
07:21:27  1 of 24 PASS not_null_daily_sales_mart_order_date .............................. [PASS in 0.12s]
07:21:27  4 of 24 PASS not_null_dim_product_categories_product_category .................. [PASS in 0.08s]
07:21:27  6 of 24 START test not_null_fact_orders_order_id ............................... [RUN]
07:21:27  7 of 24 START test not_null_fact_orders_payment_method ......................... [RUN]
07:21:27  8 of 24 START test not_null_fact_orders_product_category ....................... [RUN]
07:21:27  5 of 24 PASS not_null_fact_orders_customer_id .................................. [PASS in 0.07s]
07:21:27  9 of 24 START test not_null_stg_orders_customer_id ............................. [RUN]
07:21:27  6 of 24 PASS not_null_fact_orders_order_id ..................................... [PASS in 0.07s]
07:21:27  7 of 24 PASS not_null_fact_orders_payment_method ............................... [PASS in 0.06s]
07:21:27  10 of 24 START test not_null_stg_orders_discount_pct ........................... [RUN]
07:21:27  11 of 24 START test not_null_stg_orders_order_id ............................... [RUN]
07:21:27  8 of 24 PASS not_null_fact_orders_product_category ............................. [PASS in 0.09s]
07:21:27  12 of 24 START test not_null_stg_orders_order_ts ............................... [RUN]
07:21:27  9 of 24 PASS not_null_stg_orders_customer_id ................................... [PASS in 0.09s]
07:21:27  10 of 24 PASS not_null_stg_orders_discount_pct ................................. [PASS in 0.06s]
07:21:27  13 of 24 START test not_null_stg_orders_quantity ............................... [RUN]
07:21:27  14 of 24 START test not_null_stg_orders_status ................................. [RUN]
07:21:27  11 of 24 PASS not_null_stg_orders_order_id ..................................... [PASS in 0.07s]
07:21:27  15 of 24 START test not_null_stg_orders_unit_price ............................. [RUN]
07:21:27  12 of 24 PASS not_null_stg_orders_order_ts ..................................... [PASS in 0.07s]
07:21:27  16 of 24 START test relationships_fact_orders_customer_id__customer_id__ref_dim_customers_ [RUN]
07:21:27  13 of 24 PASS not_null_stg_orders_quantity ..................................... [PASS in 0.13s]
07:21:27  17 of 24 START test relationships_fact_orders_payment_method__payment_method__ref_dim_payment_methods_ [RUN]
07:21:27  14 of 24 PASS not_null_stg_orders_status ....................................... [PASS in 0.14s]
07:21:27  18 of 24 START test relationships_fact_orders_product_category__product_category__ref_dim_product_categories_ [RUN]
07:21:27  15 of 24 PASS not_null_stg_orders_unit_price ................................... [PASS in 0.14s]
07:21:27  19 of 24 START test unique_daily_sales_mart_order_date ......................... [RUN]
07:21:27  16 of 24 PASS relationships_fact_orders_customer_id__customer_id__ref_dim_customers_ [PASS in 0.14s]
07:21:27  20 of 24 START test unique_dim_customers_customer_id ........................... [RUN]
07:21:28  17 of 24 PASS relationships_fact_orders_payment_method__payment_method__ref_dim_payment_methods_ [PASS in 0.10s]
07:21:28  18 of 24 PASS relationships_fact_orders_product_category__product_category__ref_dim_product_categories_ [PASS in 0.08s]
07:21:28  21 of 24 START test unique_dim_payment_methods_payment_method .................. [RUN]
07:21:28  22 of 24 START test unique_dim_product_categories_product_category ............. [RUN]
07:21:28  19 of 24 PASS unique_daily_sales_mart_order_date ............................... [PASS in 0.07s]
07:21:28  23 of 24 START test unique_fact_orders_order_id ................................ [RUN]
07:21:28  20 of 24 PASS unique_dim_customers_customer_id ................................. [PASS in 0.08s]
07:21:28  24 of 24 START test unique_stg_orders_order_id ................................. [RUN]
07:21:28  21 of 24 PASS unique_dim_payment_methods_payment_method ........................ [PASS in 0.08s]
07:21:28  22 of 24 PASS unique_dim_product_categories_product_category ................... [PASS in 0.07s]
07:21:28  23 of 24 PASS unique_fact_orders_order_id ...................................... [PASS in 0.06s]
07:21:28  24 of 24 PASS unique_stg_orders_order_id ....................................... [PASS in 0.05s]
07:21:28
07:21:28  Finished running 24 data tests in 0 hours 0 minutes and 0.82 seconds (0.82s).
07:21:28
07:21:28  Completed successfully
07:21:28
07:21:28  Done. PASS=24 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=24
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse\warehouse\dbt_ecommerce>
```

48. buat dokumentasi

```
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse\warehouse\dbt_ecommerce> dbt docs generate --profiles-dir .
07:23:01  Running with dbt=1.12.6
07:23:02  Registered adapter: duckdb=1.11.0
07:23:02  Found 6 models, 24 data tests, 1 source, 500 macros
07:23:02
07:23:02  Concurrency: 4 threads (target='dev')
07:23:02
07:23:03  Building catalog
07:23:03  Catalog written to C:\Users\LEGION\Downloads\ecommerce-event-warehouse\warehouse\dbt_ecommerce\target\catalog.json
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse\warehouse\dbt_ecommerce>
```

49. verifikasi isi warehouse. Buat file baru warehouse/verify_warehouse.py:




```python
from pathlib import Path
import duckdb

BASE_DIR = Path(__file__).resolve().parent
DB_FILE = BASE_DIR / "ecommerce.duckdb"

TABLES = [
    "raw_orders",
    "stg_orders",
    "dim_customers",
    "dim_product_categories",
    "dim_payment_methods",
    "fact_orders",
    "daily_sales_mart",
]


def main():
    con = duckdb.connect(str(DB_FILE), read_only=True)

    try:
        print("=" * 55)
        print("VERIFIKASI DATA WAREHOUSE")
        print("=" * 55)

        for table in TABLES:
            count = con.execute(
                f"SELECT COUNT(*) FROM {table}"
            ).fetchone()[0]
            print(f"{table:<30} {count:>8} baris")

        print("\nRINGKASAN FAKTA TRANSAKSI")
        summary = con.execute("""
            SELECT
                COUNT(*) AS total_orders,
                COUNT(DISTINCT order_id) AS unique_orders,
                ROUND(SUM(gross_amount), 2) AS gross_sales,
                ROUND(SUM(discount_amount), 2) AS total_discount,
                ROUND(SUM(net_amount), 2) AS net_sales
            FROM fact_orders
        """).fetchone()

        print(f"Total order       : {summary[0]}")
        print(f"Order unik        : {summary[1]}")
        print(f"Penjualan bruto   : {summary[2]}")
        print(f"Total diskon      : {summary[3]}")
        print(f"Penjualan neto    : {summary[4]}")

        print("\nPREVIEW DAILY SALES MART")
        result = con.execute("""
            SELECT *
            FROM daily_sales_mart
            ORDER BY order_date
        """).df()

        print(result.to_string(index=False))

    finally:
        con.close()


if __name__ == "__main__":
    main()
```

50. jalankan di terminal

```
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> python warehouse/verify_warehouse.py
============================================================
VERIFIKASI DATA WAREHOUSE
============================================================
raw_orders                      30 baris
stg_orders                      30 baris
dim_customers                   25 baris
dim_product_categories           4 baris
dim_payment_methods              3 baris
fact_orders                     30 baris
daily_sales_mart                10 baris

RINGKASAN FAKTA TRANSAKSI
Total order       : 30
Order unik        : 30
Penjualan bruto   : 4245.83
Total diskon      : 297.37
Penjualan neto    : 3948.48

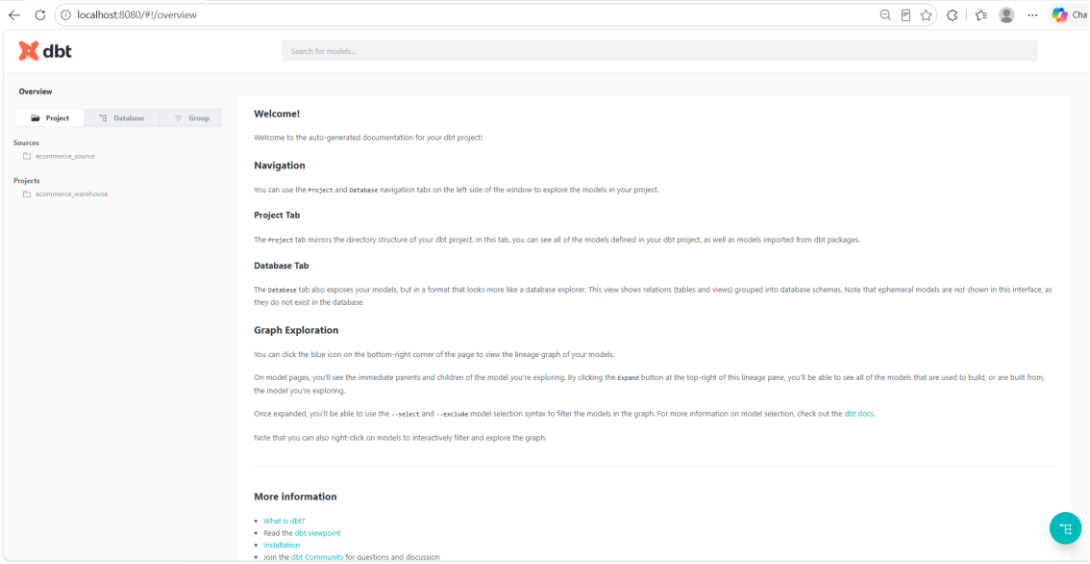
PREVIEW DAILY SALES MART
order_date  total_orders  total_customers  total_quantity  total_gross_sales  total_discount  total_net_sales  paid_orders  shi
pped_orders  cancelled_orders  refunded_orders
2026-09-01            3                3             6.0             379.97           16.40           363.57            2
          1                0                0
2026-09-02            3                3             4.0             287.90           29.09           258.82            1
          1                1                0
2026-09-03            3                3             7.0             616.50           32.45           584.05            2
          0                0                1
2026-09-04            3                3             9.0             165.00            8.40           156.60            2
          1                0                0
2026-09-05            3                3             4.0             282.00            6.40           275.60            2
          1                0                0
2026-09-06            2                2             3.0             909.00           96.40           812.60            2
          0                0                0
2026-09-07            4                4             8.0             429.96           27.50           402.46            3
          1                0                0
2026-09-08            2                2             5.0             110.50            4.13           106.38            1
          1                0                0
2026-09-09            4                4             5.0             460.00           36.90           423.10            3
          1                0                0
2026-09-10            3                3             4.0             605.00           39.70           565.30            2
          1                0                0
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse>
```

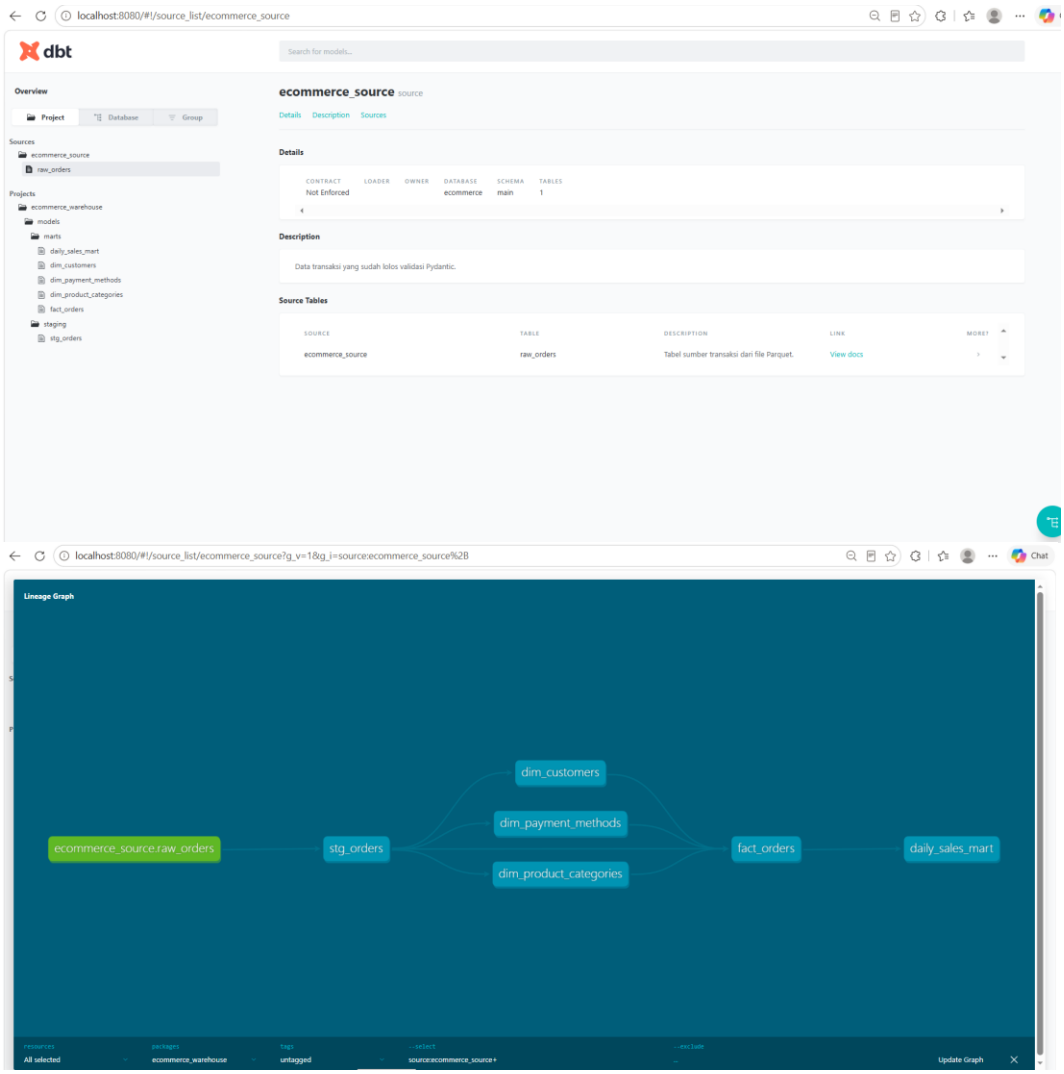
51. Buka dokumentasi dbt

```
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> cd warehouse\dbt_ecommerce
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse\warehouse\dbt_ecommerce> dbt docs serve --profiles-di
07:30:24  Running with dbt=1.12.6
Serving docs at 8080
To access from your browser, navigate to: http://localhost:8080

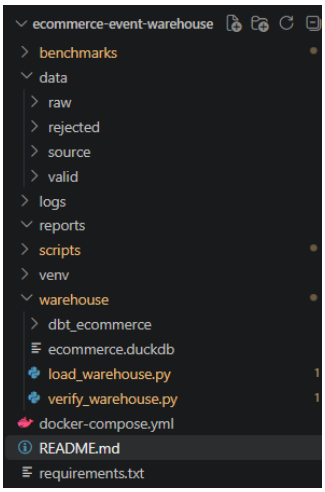


Press Ctrl+C to exit.
127.0.0.1 - - [25/Oct/2026 14:30:25] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [25/Oct/2026 14:30:26] "GET /manifest.json?cb=1792913426264 HTTP/1.1" 200 -
127.0.0.1 - - [25/Oct/2026 14:30:26] "GET /catalog.json?cb=1792913426264 HTTP/1.1" 200 -
127.0.0.1 - - [25/Oct/2026 14:30:26] code 404, message File not found
127.0.0.1 - - [25/Oct/2026 14:30:26] "GET /%7B%7B%20getIcon(item.type,%20'off')%20%7D%7D HTTP/1.1" 404 -
127.0.0.1 - - [25/Oct/2026 14:30:26] code 404, message File not found
127.0.0.1 - - [25/Oct/2026 14:30:26] "GET /%7B%7B%20getIcon(item.type,%20'on')%20%7D%7D HTTP/1.1" 404 -
127.0.0.1 - - [25/Oct/2026 14:30:28] code 404, message File not found
127.0.0.1 - - [25/Oct/2026 14:30:28] "GET /$%7Brequire('./assets/favicons/favicon.ico')%7D HTTP/1.1" 404 -
```





52. membuat README.md



End-to-End Event-to-Warehouse Pipeline: Analisis Transaksi E-Commerce

1. Deskripsi Proyek

Proyek ini merupakan Mini Project Praktikum Data Processing Workflow yang mengimplementasikan pipeline pemrosesan data transaksi e-commerce dari proses ingestion hingga penyimpanan dan analisis di data warehouse.

Pipeline menggabungkan batch ingestion, event streaming menggunakan Apache Kafka, validasi data menggunakan Pydantic, pemrosesan data menggunakan Pandas dan Polars, serta pemodelan data warehouse menggunakan DuckDB dan dbt.

Tujuan proyek ini adalah membangun alur pemrosesan data yang terstruktur, dapat diuji, mendukung penanganan data tidak valid, dan menghasilkan ringkasan penjualan harian.

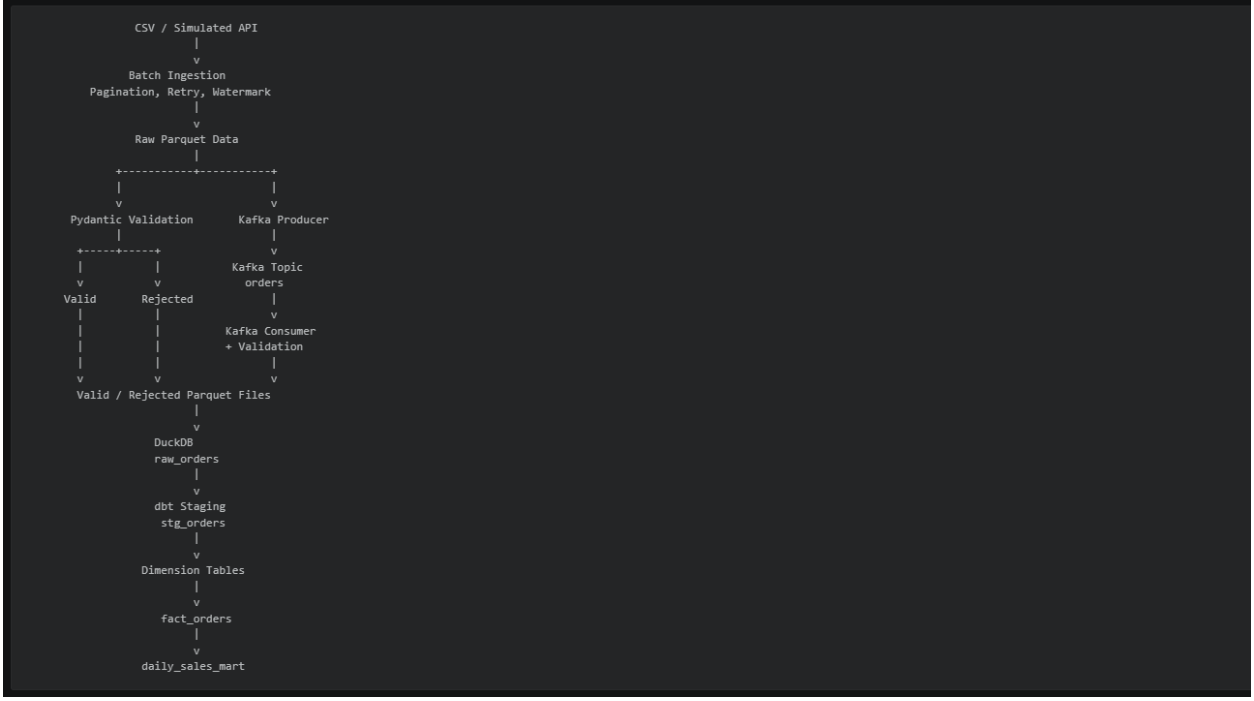
2. Tujuan

1. Mengimplementasikan batch ingestion dengan pagination, retry menggunakan exponential backoff, dan incremental loading berbasis watermark.
2. Membangun pipeline event streaming menggunakan Kafka producer dan consumer.
3. Memvalidasi kontrak data transaksi dan mengarantina data yang tidak valid.
4. Membandingkan performa Pandas dan Polars, termasuk Polars Lazy API.
5. Mengeksplorasi pengaruh distribusi data skew terhadap waktu pemrosesan.
6. Membangun data warehouse menggunakan DuckDB dan transformasi data menggunakan dbt.
7. Menghasilkan daily sales mart serta melakukan pengujian kualitas data.

3. Teknologi yang Digunakan

Teknologi	Fungsi
Python	Bahasa pemrograman utama
Pandas	Pemrosesan dan analisis data
Polars	Pemrosesan data eager dan lazy
Pydantic	Validasi kontrak data
Apache Kafka	Pengiriman dan konsumsi event transaksi
Docker Compose	Menjalankan layanan Kafka
PyArrow / Parquet	Penyimpanan data berbasis kolom
DuckDB	Database analitik lokal
dbt-duckdb	Transformasi, pengujian, dan dokumentasi model data
Requests	Dukungan HTTP untuk ingestion
Matplotlib / library visualisasi tambahan	Dapat digunakan untuk visualisasi hasil analisis jika diperlukan

4. Arsitektur Pipeline



5. Struktur Folder

```
ecommerce-event-warehouse/
├── data/
│   ├── source/
│   │   └── orders.csv
│   ├── raw/
│   │   ├── orders_raw.parquet
│   │   └── watermark.json
│   ├── valid/
│   │   ├── orders_valid.parquet
│   │   └── orders_kafka_valid.parquet
│   └── rejected/
│       ├── orders_rejected.parquet
│       ├── orders_rejected.csv
│       ├── orders_kafka_rejected.parquet
│       └── orders_kafka_rejected.csv
├── scripts/
│   ├── batch_ingest.py
│   ├── validate_orders.py
│   ├── kafka_producer.py
│   └── kafka_consumer.py
├── benchmarks/
│   ├── benchmark_pandas_polars.py
│   ├── benchmark_skew.py
│   ├── benchmark_results.csv
│   └── skew_results.csv
├── warehouse/
│   ├── load_warehouse.py
│   ├── verify_warehouse.py
│   ├── ecommerce.duckdb
│   └── dbt_ecommerce/
│       ├── dbt_project.yml
│       ├── profiles.yml
│       ├── models/
│       └── target/
├── docker-compose.yml
├── requirements.txt
└── README.md
```

Struktur di atas merupakan gambaran artefak utama. File tambahan dapat muncul selama proses pengujian dan pembuatan dokumentasi dbt.

6. Persiapan Lingkungan

Persyaratan lingkungan:

- Python 3.10 atau versi yang kompatibel dengan dependensi proyek.
- Docker Desktop dengan Docker Compose.
- Visual Studio Code atau editor kode lainnya.

6.1. Membuat dan mengaktifkan virtual environment

Jalankan dari folder utama proyek:

```
python -m venv venv
.\venv\Scripts\Activate.ps1
```

6.2. Menginstal dependensi

```
python -m pip install -r requirements.txt
```

6.3. Menjalankan Kafka

Pastikan Docker Desktop sedang berjalan, kemudian jalankan:

```
docker compose up -d
docker compose ps
```

Pastikan container Kafka berjalan dalam kondisi sehat sebelum producer dan consumer digunakan.

7. Implementasi Pipeline

7.1. Batch Ingestion

Batch ingestion diimplementasikan melalui `scripts/batch_ingest.py`.

Fitur yang diterapkan:

- Pagination dengan ukuran halaman 10 data.
- Simulasi kegagalan sementara pada salah satu halaman.
- Retry dengan exponential backoff.
- Incremental loading berdasarkan watermark timestamp.
- Penyimpanan data dalam format Parquet.
- Deduplikasi berdasarkan `order_id`.

Data sumber berisi 36 baris. Pada eksekusi pertama, 36 data berhasil dimuat ke raw Parquet. Pada eksekusi kedua terhadap sumber statis yang sama, tidak ada data baru yang ditambahkan.

Hasil ini menunjukkan bahwa mekanisme deduplikasi mencegah penambahan ulang transaksi yang sama pada pengujian tersebut.

7.2. Validasi Data Menggunakan Pydantic

Validasi dilakukan melalui `scripts/validate_orders.py`.

Aturan validasi meliputi:

- `order_id` dan `customer_id` wajib terisi.
- `quantity` harus berupa integer positif.
- `unit_price` tidak boleh negatif.
- `discount_pct` harus berada pada rentang 0 sampai 1.
- `order_ts` harus dapat diparse sebagai timestamp.
- `status` harus termasuk `paid`, `shipped`, `cancelled`, atau `refunded`.

Data valid dan data yang ditolak disimpan secara terpisah. File rejected juga menyimpan alasan penolakan agar kesalahan data dapat ditelusuri.

7.3. Event Streaming dengan Kafka

Kafka digunakan untuk mengirim event transaksi melalui topic `orders`.

Producer:

```
python scripts/kafka_producer.py
```

Consumer:

```
python scripts/kafka_consumer.py
```

Topic `orders` dikonfigurasi menggunakan tiga partisi. `order_id` digunakan sebagai key pesan untuk membantu pengelompokan pesan berdasarkan kunci.

Consumer menggunakan consumer group `orders-validation-group`, melakukan validasi event, menyimpan data valid dan rejected, serta melakukan commit offset setelah proses penyimpanan.

Pipeline ini menggunakan pendekatan at-least-once dengan sink yang dirancang idempoten untuk mengurangi duplikasi saat event diproses ulang.

Konfigurasi Kafka lokal menggunakan satu broker dan replication factor 1. Konfigurasi ini ditujukan untuk praktikum, bukan untuk menjamin ketersediaan tinggi pada lingkungan produksi.

7.4. Benchmark Pandas dan Polars

Benchmark dilakukan melalui:

```
python benchmarks/benchmark_pandas_polars.py
```

Pengujian membandingkan:

- Pandas eager.
- Polars eager.
- Polars lazy.

Hasil benchmark disimpan pada `benchmarks/benchmark_results.csv`.

Pengujian tambahan terhadap distribusi data seimbang dan skew dilakukan melalui:

```
python benchmarks/benchmark_skew.py
```

Dataset eksperimen berisi 200.000 baris untuk masing-masing distribusi. Hasil disimpan pada `benchmarks/skew_results.csv`.

Hasil pengukuran perlu diinterpretasikan sesuai ukuran dataset, kondisi perangkat, jumlah pengulangan, dan overhead pembacaan file.

7.5. DuckDB Data Warehouse

Data valid dimuat ke DuckDB melalui:

```
python warehouse/load_warehouse.py
```

Database disimpan pada `warehouse/ecommerce.duckdb`, dengan tabel awal `raw_orders`.

Pemeriksaan tabel dan ringkasan transaksi dilakukan menggunakan:

```
python warehouse/verify_warehouse.py
```

7.6. Transformasi Data Menggunakan dbt

Masuk ke direktori proyek dbt:

```
cd warehouse/dbt_ecommerce
```

Periksa konfigurasi koneksi:

```
dbt debug --profiles-dir .
```

Jalankan transformasi:

```
dbt run --profiles-dir .
```

Jalankan pengujian kualitas data:

```
dbt test --profiles-dir .
```

Buat dokumentasi:

```
dbt docs generate --profiles-dir .
```

Jalankan server dokumentasi:

```
dbt docs serve --profiles-dir .
```

Dokumentasi dapat diakses melalui `http://localhost:8080` selama server berjalan.

Model dbt mencakup:

- `stg_orders`: staging dan standardisasi data transaksi.
- `dim_customers`: dimensi pelanggan.
- `dim_product_categories`: dimensi kategori produk.

8. Hasil dan Evaluasi

8.1. Hasil Ingestion dan Validasi

Tahap	Jumlah data
Data sumber	36
Data valid	30
Data rejected	6
Data raw setelah ingestion pertama	36
Data baru pada ingestion kedua	0

Enam data rejected mencakup kasus quantity nol, harga negatif, diskon di luar rentang, timestamp tidak valid, status tidak dikenal, dan customer ID kosong.

8.2. Hasil Benchmark Pandas vs. Polars

Pengukuran pada dataset raw berukuran 36 baris menghasilkan rata-rata waktu berikut:

Metode	Rata-rata waktu
Pandas eager	0,008101 detik
Polars eager	0,001927 detik
Polars lazy	0,001804 detik

Dalam pengujian ini, Polars eager sekitar 4,2 kali lebih cepat dan Polars lazy sekitar 4,5 kali lebih cepat dibandingkan Pandas eager.

Namun, dataset hanya berisi 36 baris sehingga hasil tersebut tidak dapat digeneralisasikan sebagai perbandingan performa pada semua ukuran data.

8.3. Hasil Eksperimen Data Skew

Eksperimen tambahan menggunakan dataset 200.000 baris dengan 20 kategori.

Distribusi	Proporsi kategori terbesar
Balanced	5,08%
Skewed	95,04%

Pada pengujian ini, waktu pemrosesan tidak selalu meningkat pada data skew. Hasil pengukuran dapat dipengaruhi oleh overhead I/O, implementasi agregasi, dan variasi lingkungan eksekusi.

8.4. Hasil Data Warehouse

Tabel	Jumlah baris
`raw_orders`	30
`stg_orders`	30
`dim_customers`	25
`dim_product_categories`	4
`dim_payment_methods`	3
`fact_orders`	30
`daily_sales_mart`	10

8.5. Ringkasan Penjualan

Metrik	Hasil
Total transaksi	30
Transaksi unik	30
Penjualan bruto	4.245,83
Total diskon	297,37
Penjualan neto	3.948,48

Seluruh enam model dbt berhasil dibuat dan 24 data test dbt berhasil lulus pada eksekusi pengujian yang dilakukan.

9. Keterbatasan

1. Sumber batch menggunakan CSV dan simulasi pagination API, bukan integrasi dengan API eksternal produksi.
2. Retry diuji menggunakan kegagalan yang disimulasikan.
3. Watermark timestamp perlu dikembangkan lebih lanjut untuk menangani late-arriving events dan perubahan data historis.
4. Kafka berjalan pada satu broker dengan replication factor 1.
5. Benchmark awal menggunakan dataset yang sangat kecil, sedangkan eksperimen skew menggunakan dataset buatan.
6. Hasil waktu benchmark bergantung pada perangkat dan kondisi eksekusi.

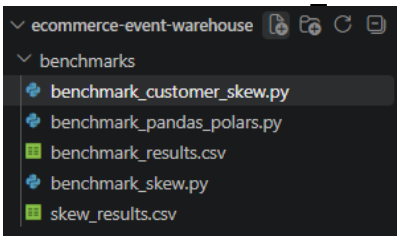
10. Kesimpulan

Proyek ini berhasil membangun pipeline pemrosesan transaksi e-commerce dari ingestion, validasi, event streaming, hingga data warehouse dan analisis penjualan harian.

Pengujian menunjukkan bahwa mekanisme validasi dapat memisahkan data valid dan tidak valid, ingestion kedua tidak menambahkan duplikasi pada sumber statis yang diuji, serta model warehouse berhasil dibangun dan n
data dbt.

Pipeline ini menjadi dasar untuk pengembangan lanjutan seperti integrasi API sesungguhnya, pemantauan pipeline, penanganan late-arriving data, dan penerapan arsitektur Kafka multi-broker.

53. Lengkapi kodingan eksperimen data skew
Beri nama benchmark_customer_skew.py.



```python

from pathlib import Path
import time

import pandas as pd
import polars as pl


# ============================================================
# 1. KONFIGURASI
# ============================================================
BASE_DIR = Path(__file__).resolve().parent.parent
DATA_DIR = BASE_DIR / "data" / "source"
RESULT_DIR = BASE_DIR / "benchmarks"

DATA_DIR.mkdir(parents=True, exist_ok=True)

N_ROWS = 200_000
N_CUSTOMERS = 1_000
N_REPEATS = 5
DOMINANT_CUSTOMER = "CUST_DOMINANT"


# ============================================================
# 2. MEMBUAT DATA BALANCED DAN SKEWED
# ============================================================
def create_datasets():
    # Data balanced: pesanan tersebar ke banyak customer.
    balanced = pd.DataFrame({
        "order_id": [f"ORD{i:07d}" for i in range(N_ROWS)],
        "customer_id": [
            f"CUST{i % N_CUSTOMERS:04d}"
            for i in range(N_ROWS)
        ],
        "net_amount": [
            float((i % 100) + 1)
            for i in range(N_ROWS)
        ],
    })

    # Data skewed: 95% pesanan dimiliki satu customer.
    dominant_count = int(N_ROWS * 0.95)
    other_count = N_ROWS - dominant_count

    skewed_customer_ids = (
        [DOMINANT_CUSTOMER] * dominant_count
        + [
            f"CUST{i % (N_CUSTOMERS - 1):04d}"
            for i in range(other_count)
        ]
    )

    skewed = pd.DataFrame({
        "order_id": [f"ORD{i:07d}" for i in range(N_ROWS)],
        "customer_id": skewed_customer_ids,
        "net_amount": [
            float((i % 100) + 1)
            for i in range(N_ROWS)
        ],
    })

    balanced_path = DATA_DIR / "customer_balanced.parquet"
    skewed_path = DATA_DIR / "customer_skewed.parquet"

    balanced.to_parquet(balanced_path, index=False)
    skewed.to_parquet(skewed_path, index=False)

    print(f"Data balanced: {len(balanced):,} baris")
    print(f"Data skewed  : {len(skewed):,} baris")
    print(f"Customer dominan: {DOMINANT_CUSTOMER}")
    print(f"Proporsi dominan: {dominant_count / N_ROWS:.0%}")

    return balanced_path, skewed_path


# ============================================================
# 3. BENCHMARK PANDAS, POLARS EAGER, DAN POLARS LAZY
# ============================================================
def benchmark_file(path, distribution):
    results = []

    for method in ["Pandas Eager", "Polars Eager", "Polars Lazy"]:
        times = []

        for _ in range(N_REPEATS):
            start = time.perf_counter()

            if method == "Pandas Eager":
                df = pd.read_parquet(path)
                result = (
                    df.groupby("customer_id", as_index=False)
                    .agg(
                        total_orders=("order_id", "count"),
                        total_net_sales=("net_amount", "sum"),
                    )
                )

            elif method == "Polars Eager":
                df = pl.read_parquet(path)
                result = (
                    df.group_by("customer_id")
                    .agg(
                        pl.col("order_id").count().alias("total_orders"),
                        pl.col("net_amount").sum().alias("total_net_sales"),
                    )
                )

            else:
                result = (
                    pl.scan_parquet(path)
                    .group_by("customer_id")
                    .agg(
                        pl.col("order_id").count().alias("total_orders"),
                        pl.col("net_amount").sum().alias("total_net_sales"),
                    )
                    .collect()
                )
```

```
            elapsed = time.perf_counter() - start
            times.append(elapsed)

        results.append({
            "distribution": distribution,
            "method": method,
            "rows": N_ROWS,
            "groups": len(result),
            "mean_seconds": sum(times) / len(times),
            "min_seconds": min(times),
            "max_seconds": max(times),
        })

    return results


# ============================================================
# 4. PERIKSA DISTRIBUSI CUSTOMER
# ============================================================
def show_distribution(path, distribution):
    df = pd.read_parquet(path)

    counts = df["customer_id"].value_counts()
    total_rows = len(df)

    print(f"\n--- Distribusi {distribution} ---")
    print(f"Jumlah baris: {total_rows:,}")
    print(f"Jumlah customer unik: {df['customer_id'].nunique():,}")
    print(f"Customer dengan pesanan terbanyak: {counts.index[0]}")
    print(f"Jumlah pesanan terbanyak: {counts.iloc[0]:,}")
    print(f"Proporsi pesanan terbanyak: {counts.iloc[0] / total_rows:.2%}")

    print("\nLima customer teratas:")
    print(counts.head(5).to_string())


# ============================================================
# 5. JALANKAN EKSPERIMEN DAN SIMPAN HASIL
# ============================================================
def main():
    balanced_path, skewed_path = create_datasets()

    show_distribution(balanced_path, "Balanced")
    show_distribution(skewed_path, "Skewed")

    results = []
    results.extend(benchmark_file(balanced_path, "Balanced"))
    results.extend(benchmark_file(skewed_path, "Skewed"))

    results_df = pd.DataFrame(results)
    output_path = RESULT_DIR / "customer_skew_results.csv"
    results_df.to_csv(output_path, index=False)

    print("\n--- HASIL BENCHMARK ---")
    print(
        results_df[
            ["distribution", "method", "rows", "groups", "mean_seconds"]
        ].to_string(index=False)
    )

    print(f"\nHasil tersimpan di: {output_path}")


if __name__ == "__main__":
    main()
```

54. Jalankan di terminal

```
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> python benchmarks/benchmark_customer_skew.py
Customer dominan: CUST_DOMINANT
Proporsi dominan: 95%

--- Distribusi Balanced ---
Jumlah baris: 200,000
Jumlah customer unik: 1,000
Customer dengan pesanan terbanyak: CUST0999
Jumlah pesanan terbanyak: 200
Proporsi pesanan terbanyak: 0.10%

Lima customer teratas:
customer_id
CUST0999    200
CUST0000    200
CUST0001    200
CUST0002    200
CUST0003    200

--- Distribusi Skewed ---
Jumlah baris: 200,000
Jumlah customer unik: 1,000
Customer dengan pesanan terbanyak: CUST_DOMINANT
Jumlah pesanan terbanyak: 190,000
Proporsi pesanan terbanyak: 95.00%

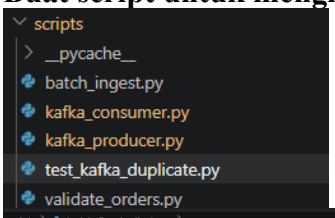
Lima customer teratas:
customer_id
CUST_DOMINANT    190000
CUST0004             11
CUST0008             11
CUST0007             11
CUST0009             11

--- HASIL BENCHMARK ---
distribution       method   rows  groups  mean_seconds
    Balanced Pandas Eager 200000    1000      0.072272
    Balanced Polars Eager 200000    1000      0.038876
    Balanced  Polars Lazy 200000    1000      0.010416
      Skewed Pandas Eager 200000    1000      0.092139
      Skewed Polars Eager 200000    1000      0.015249
      Skewed  Polars Lazy 200000    1000      0.013100

Hasil tersimpan di: C:\Users\LEGION\Downloads\ecommerce-event-warehouse\benchmarks\customer_skew_results.csv
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse>
```

55. demonstrasi duplikasi event Kafka

Buat script untuk mengirim duplikat




```python
from pathlib import Path
import json

import pandas as pd
from confluent_kafka import Producer


BASE_DIR = Path(__file__).resolve().parent.parent
SOURCE_FILE = BASE_DIR / "data" / "source" / "orders.csv"

BOOTSTRAP_SERVERS = "localhost:9092"
TOPIC = "orders"


def delivery_report(error, message):
    if error is not None:
        print(f"[GAGAL] {error}")
    else:
        print(
            f"[TERKIRIM] order_id={message.key().decode('utf-8')} "
            f"partition={message.partition()} "
            f"offset={message.offset()}"
        )


def main():
    if not SOURCE_FILE.exists():
        raise FileNotFoundError(f"File tidak ditemukan: {SOURCE_FILE}")

    df = pd.read_csv(SOURCE_FILE)

    # Ambil satu transaksi asli agar payload tetap valid.
    row = df.iloc[0].to_dict()
    order_id = str(row["order_id"])

    # Ubah NaN menjadi None agar dapat dikirim sebagai JSON.
    clean_row = {
        key: None if pd.isna(value) else value
        for key, value in row.items()
    }

    payload = json.dumps(clean_row, ensure_ascii=False).encode("utf-8")
    key = order_id.encode("utf-8")

    producer = Producer({
        "bootstrap.servers": BOOTSTRAP_SERVERS,
        "client.id": "duplicate-test-producer",
    })

    print(f"Menguji duplikasi order_id: {order_id}")

    # Kirim payload identik dua kali dengan key yang sama.
    for attempt in range(1, 3):
        producer.produce(
            TOPIC,
            key=key,
            value=payload,
            callback=delivery_report,
        )
        producer.poll(0)
        print(f"Event duplikat ke-{attempt} dimasukkan ke antrean")

    remaining = producer.flush(30)

    if remaining > 0:
        raise RuntimeError(f"{remaining} event belum selesai terkirim")

    print("\nPengujian producer selesai.")
    print("Dua event dengan order_id yang sama telah dikirim.")


if __name__ == "__main__":
    main()
```

56. jalankan di terminal

```
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> python scripts/test_kafka_duplicate.py
Menguji duplikasi order_id: O1001
Event duplikat ke-1 dimasukkan ke antrean
Event duplikat ke-2 dimasukkan ke antrean
[TERKIRIM] order_id=O1001 partition=2 offset=11
[TERKIRIM] order_id=O1001 partition=2 offset=12

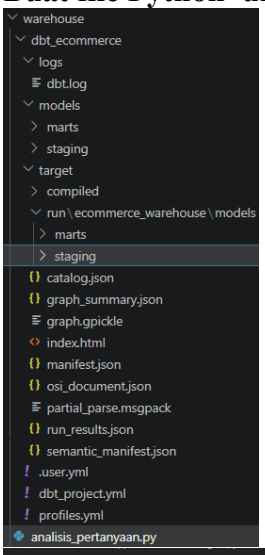
Pengujian producer selesai.
Dua event dengan order_id yang sama telah dikirim.
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse>
```

57. jalankan consumer:

```
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> python scripts/kafka_consumer.py
Mendengarkan topic 'orders'...
Consumer group: orders-validation-group
Menunggu pesan; akan berhenti setelah 5 detik idle.
[VALID] O1001 (partition=2, offset=11)
[VALID] O1001 (partition=2, offset=12)

============================================
HASIL KAFKA CONSUMER
============================================
Pesan diterima sesi ini : 2
Event valid baru        : 2
Event rejected baru     : 0
Total valid di sink     : 30
Total rejected di sink  : 6
Valid Parquet           : data\valid\orders_kafka_valid.parquet
Rejected Parquet        : data\rejected\orders_kafka_rejected.parquet
Rejected CSV            : data\rejected\orders_kafka_rejected.csv
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse>
```

58. Buat file Python untuk menjawab pertanyaan



warehouse > analisis_pertanyaan.py > ...

```python
import duckdb

# Membuka database DuckDB yang sudah dibuat
con = duckdb.connect(
    "warehouse/ecommerce.duckdb",
    read_only=True
)

queries = {
    "2. Penjualan Bersih per Kategori": """
        SELECT
            product_category,
            ROUND(SUM(net_amount), 2) AS total_penjualan_bersih
        FROM main.stg_orders
        GROUP BY product_category
        ORDER BY total_penjualan_bersih DESC
    """,

    "3. Jumlah dan Nilai Pesanan per Status": """
        SELECT
            status,
            COUNT(*) AS jumlah_pesanan,
            ROUND(SUM(net_amount), 2) AS total_penjualan_bersih
        FROM main.stg_orders
        GROUP BY status
        ORDER BY jumlah_pesanan DESC
    """,

    "4. Metode Pembayaran Terpopuler": """
        SELECT
            payment_method,
            COUNT(*) AS jumlah_pesanan
        FROM main.stg_orders
        GROUP BY payment_method
        ORDER BY jumlah_pesanan DESC
    """
}

for judul, query in queries.items():
    print("\n" + "=" * 60)
    print(judul)
    print("=" * 60)

    hasil = con.execute(query).df()
    print(hasil.to_string(index=False))

con.close()
```

59. Jalankan di terminal

```
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse> python warehouse/analisis_pertanyaan.py

============================================================
2. Penjualan Bersih per Kategori
============================================================
product_category  total_penjualan_bersih
     Electronics                2079.17
            Home                 996.98
         Fashion                 635.60
           Books                 236.73

============================================================
3. Jumlah dan Nilai Pesanan per Status
============================================================
   status  jumlah_pesanan  total_penjualan_bersih
     paid              20                 2443.85
  shipped               8                  968.18
 refunded               1                  474.05
cancelled               1                   62.40

============================================================
4. Metode Pembayaran Terpopuler
============================================================
payment_method  jumlah_pesanan
          card              14
       ewallet               8
      transfer               8
(venv) PS C:\Users\LEGION\Downloads\ecommerce-event-warehouse>
```

60. Pertanyaan Analitik Minimal — E-Commerce Event-to-Warehouse Pipeline

1. Berapa nilai penjualan bersih per hari?

Berdasarkan tabel daily_sales_mart, diperoleh nilai penjualan bersih selama periode 1–10 September 2026 sebagai berikut.

Tanggal	Penjualan bersih
1 September 2026	363,57
2 September 2026	258,82
3 September 2026	584,05
4 September 2026	156,60
5 September 2026	275,60
6 September 2026	812,60
7 September 2026	402,46
8 September 2026	106,38
9 September 2026	423,10

10 September 2026	565,30
Total	3.948,48

Analisis: Penjualan bersih tertinggi terjadi pada 6 September 2026 sebesar 812,60, sedangkan penjualan terendah terjadi pada 8 September 2026 sebesar 106,38. Dari 30 transaksi valid, total penjualan bruto sebesar 4.245,83 dikurangi diskon sebesar 297,37 menghasilkan penjualan bersih sebesar 3.948,48. Satuan mata uang tidak ditentukan secara eksplisit dalam dataset.

2. Kategori produk mana yang memiliki penjualan bersih tertinggi?

Hasil agregasi net_amount berdasarkan product_category menunjukkan hasil berikut.

Kategori produk	Penjualan bersih
Electronics	2.079,17
Home	996,98
Fashion	635,60
Books	236,73
Total	3.948,48

Analisis: Kategori Electronics memiliki penjualan bersih tertinggi, yaitu 2.079,17, atau sekitar 52,66% dari total penjualan bersih. Kategori Home berada di posisi kedua dengan 996,98, sedangkan Books memiliki kontribusi terendah sebesar 236,73. Hasil ini menunjukkan bahwa kategori Electronics menjadi kontributor pendapatan bersih terbesar dalam dataset yang dianalisis.

3. Bagaimana jumlah dan nilai pesanan menurut status?

Berdasarkan hasil query pada tabel stg_orders, distribusi pesanan menurut status adalah sebagai berikut.

Status	Jumlah pesanan	Penjualan bersih
paid	20	2.443,85
shipped	8	968,18
refunded	1	474,05
cancelled	1	62,40

Total	30	3.948,48

Analisis: Status paid memiliki jumlah transaksi terbanyak, yaitu 20 pesanan, dengan total nilai penjualan bersih 2.443,85. Status shipped mencakup 8 pesanan dengan nilai 968,18, sedangkan status refunded dan cancelled masing-masing memiliki satu pesanan. Hasil ini menggambarkan distribusi nilai transaksi berdasarkan status yang tercatat. Namun, nilai tersebut merupakan agregasi net_amount, sehingga tidak otomatis menunjukkan pendapatan yang sudah direalisasikan atau dampak pengembalian dana secara akuntansi.

4. Metode pembayaran mana yang paling sering digunakan?

Hasil penghitungan transaksi berdasarkan payment_method adalah sebagai berikut.

Metode pembayaran	Jumlah pesanan	Persentase
card	14	46,67%
ewallet	8	26,67%
transfer	8	26,67%
Total	30	100%

Analisis: Metode pembayaran card merupakan metode yang paling sering digunakan, dengan 14 transaksi atau 46,67% dari seluruh transaksi valid. Metode ewallet dan transfer masing-masing digunakan dalam 8 transaksi atau 26,67%. Temuan ini menunjukkan bahwa card menjadi metode pembayaran paling dominan dalam dataset yang diamati.

5. Berapa banyak record yang ditolak oleh data contract, dan apa penyebab terbesarnya?

Validasi menggunakan Pydantic menghasilkan 30 record valid dan 6 record rejected dari total 36 record sumber.

ID transaksi	Penyebab penolakan
O1031	Kuantitas tidak boleh nol atau negatif
O1032	Harga satuan tidak boleh negatif
O1033	Persentase diskon melebihi batas maksimum 1
O1034	Format timestamp tidak valid
O1035	Status pending tidak termasuk status yang diizinkan
O1036	customer_id kosong

Analisis: Sebanyak 6 dari 36 record ditolak, sehingga tingkat penolakan data mencapai 16,67%. Setiap penyebab muncul satu kali, sehingga tidak ada satu penyebab yang paling dominan. Data yang ditolak disimpan secara terpisah beserta alasan penolakannya agar dapat ditelusuri dan tidak masuk ke warehouse sebagai transaksi valid.

Kesimpulan analitik

Berdasarkan hasil pengolahan data, kategori Electronics memberikan kontribusi penjualan bersih terbesar sebesar 2.079,17, dan metode pembayaran card menjadi metode yang paling banyak digunakan dengan 14 transaksi. Penjualan bersih tertinggi per hari tercatat pada 6 September 2026 sebesar 812,60. Dari sisi kualitas data, enam record ditolak oleh data contract karena melanggar aturan validasi. Pemisahan data valid dan rejected membantu menjaga agar analisis warehouse hanya menggunakan transaksi yang memenuhi kontrak data.

Catatan penting: Semua nilai di atas mengikuti hasil query dan output yang sudah kamu jalankan. Sebelum laporan final dikumpulkan, pastikan angka tersebut sesuai dengan kondisi database terakhir, terutama setelah pengujian duplikasi Kafka.

Rangkuman Analisis Proyek

1. Tujuan dan gambaran umum proyek

- Latar belakang proyek: Proyek ini dibuat untuk menerapkan alur pengolahan data transaksi e-commerce secara end-to-end, mulai dari pengambilan data, validasi kualitas data, pemrosesan event secara streaming, pengujian performa pemrosesan data, hingga penyimpanan dan analisis data di dalam data warehouse.

- Tujuan utama: Membangun pipeline yang dapat mengelola data transaksi secara terstruktur, memisahkan data valid dan tidak valid, mencegah duplikasi, membandingkan performa Pandas dan Polars, serta menghasilkan data yang siap digunakan untuk analisis penjualan.

- Teknologi yang digunakan: Python untuk pemrosesan data, Pandas dan Polars untuk analisis serta benchmark, Pydantic untuk validasi, Apache Kafka untuk event streaming, Docker Compose untuk menjalankan Kafka, Parquet sebagai format penyimpanan data, DuckDB sebagai database warehouse, serta dbt untuk transformasi, pengujian, dan dokumentasi data.

- Gambaran alur sistem: Data transaksi dibaca melalui proses batch ingestion, disimpan dalam format Parquet, lalu divalidasi menggunakan Pydantic. Data juga dikirim melalui Kafka untuk diproses oleh consumer. Selanjutnya, data valid dimasukkan ke DuckDB dan ditransformasikan menggunakan dbt menjadi tabel staging, tabel dimensi, tabel fakta, dan mart penjualan harian.

1. Data Source

CSV transaksi e-commerce — 36 baris

2. Batch Ingestion

Pagination, retry, watermark, deduplikasi, Parquet

3. Validasi Data

Pydantic — 30 data valid dan 6 data ditolak

4. Event Streaming

Kafka producer → topic orders → consumer

5. Data Warehouse

DuckDB → dbt staging → dimensi dan fakta → mart

6. Hasil Analisis

Ringkasan penjualan, benchmark performa, pengujian, dan dokumentasi

2. Persiapan lingkungan kerja

- Pembuatan project: Seluruh pekerjaan dilakukan menggunakan Visual Studio Code pada folder ecommerce-event-warehouse. Struktur folder dipisahkan berdasarkan fungsi, seperti data/source, data/raw, data/valid, data/rejected, scripts, warehouse, benchmarks, dan reports. Pemisahan ini membantu menjaga kode, data, hasil pengujian, dan dokumentasi tetap terorganisasi.

- Virtual environment: Python dijalankan melalui virtual environment agar dependensi proyek terisolasi dari instalasi Python utama. Library yang digunakan antara lain pandas, pyarrow, pydantic, polars, duckdb, confluent-kafka, dan dbt-duckdb.

- Docker dan Kafka: Docker Desktop digunakan untuk menjalankan Apache Kafka tanpa perlu memasang broker secara manual di sistem operasi. Konfigurasi Kafka didefinisikan dalam docker-compose.yml, sehingga layanan dapat dinyalakan dan dihentikan menggunakan perintah Docker Compose.

3. Batch ingestion dan incremental loading

- Pengambilan data dengan pagination: Script scripts/batch_ingest.py membaca data transaksi dalam beberapa halaman dengan ukuran 10 baris per halaman. Dari total 36 baris, data dibaca melalui empat halaman, yaitu 10, 10, 10, dan 6 baris. Mekanisme pagination menggambarkan bagaimana data berukuran besar dapat diambil secara bertahap melalui API.

- Retry dengan exponential backoff: Pada pengujian, terjadi kegagalan sementara yang disimulasikan ketika membaca halaman kedua. Sistem kemudian melakukan percobaan ulang dengan jeda yang meningkat. Mekanisme ini membantu pipeline menghadapi gangguan sementara tanpa langsung menghentikan keseluruhan proses.

- Watermark: Timestamp transaksi terakhir disimpan dalam data/raw/watermark.json. Watermark yang tercatat adalah 2026-09-11T15:00:00+00:00. Nilai tersebut digunakan sebagai penanda progres incremental loading agar sistem dapat mengetahui batas waktu data yang sudah diproses.

- Penyimpanan Parquet dan deduplikasi: Data hasil ingestion disimpan dalam data/raw/orders_raw.parquet. Deduplikasi dilakukan menggunakan order_id sebagai identitas transaksi. Pada proses pertama, 36 data baru berhasil ditambahkan. Ketika script dijalankan kembali dengan sumber data yang sama, tidak ada data baru yang ditambahkan dan total data tetap 36 baris.

- Analisis hasil ingestion: Pengujian tersebut menunjukkan bahwa mekanisme deduplikasi mampu mencegah penambahan ulang transaksi yang sudah ada pada skenario pengujian. Namun, sumber data yang digunakan masih berupa CSV yang mensimulasikan API, bukan API eksternal sungguhan. Selain itu, watermark berbasis waktu transaksi perlu dirancang lebih lanjut untuk menangani data terlambat masuk atau transaksi yang diperbarui dengan timestamp yang sama.

4. Validasi data menggunakan Pydantic

- Tujuan validasi: Sebelum data dipakai untuk analisis, setiap transaksi diperiksa berdasarkan kontrak data atau data contract. Validasi diperlukan agar data yang masuk ke proses berikutnya memiliki struktur dan nilai yang sesuai dengan ketentuan.

- Aturan validasi: order_id dan customer_id wajib berupa teks yang tidak kosong. quantity harus berupa bilangan bulat positif, unit_price tidak boleh negatif, discount_pct harus berada dalam rentang 0 sampai 1, order_ts harus dapat dibaca sebagai timestamp, dan status hanya boleh bernilai paid, shipped, cancelled, atau refunded.

- Hasil validasi: Dari 36 transaksi sumber, sebanyak 30 transaksi dinyatakan valid dan 6 transaksi ditolak. Keenam data yang ditolak memiliki permasalahan berikut:

ID transaksi	Permasalahan
O1031	quantity = 0
O1032	unit_price = -12
O1033	discount_pct = 1.20
O1034	order_ts bukan timestamp yang valid
O1035	status = pending tidak termasuk status yang diizinkan
O1036	customer_id kosong

- Penyimpanan hasil validasi: Data valid disimpan dalam data/valid/orders_valid.parquet, sedangkan data yang ditolak disimpan dalam data/rejected/orders_rejected.parquet dan data/rejected/orders_rejected.csv. File penolakan juga menyimpan alasan kesalahan agar data bermasalah dapat diperiksa dan diperbaiki.

- Analisis: Hasil ini menunjukkan bahwa validasi tidak sekadar memeriksa keberadaan kolom, tetapi juga memastikan nilai transaksi memenuhi aturan bisnis. Pemisahan data valid dan rejected membuat kesalahan lebih mudah ditelusuri tanpa mencampurkan transaksi bermasalah ke dalam analisis penjualan.

5. Event streaming menggunakan Apache Kafka

- Tujuan Kafka: Berbeda dari batch ingestion yang mengambil data secara bertahap dari sumber, Kafka digunakan untuk merepresentasikan aliran event transaksi. Arsitektur producer dan consumer memungkinkan pengiriman serta pemrosesan event dipisahkan sehingga keduanya dapat dikembangkan secara independen.

- Konfigurasi topic: Kafka dijalankan menggunakan Docker Compose dan memiliki topic bernama orders dengan tiga partisi. Partisi memungkinkan event dibagi ke beberapa jalur penyimpanan dan pemrosesan. Kafka dijalankan dengan replication factor 1, sehingga konfigurasi ini cocok untuk demonstrasi lokal tetapi belum menyediakan redundansi antarbroker.

- Kafka producer: Script scripts/kafka_producer.py membaca 36 transaksi dari file sumber, mengubah setiap transaksi menjadi JSON, lalu mengirimkannya ke topic orders. order_id digunakan sebagai key event. Pengujian menunjukkan bahwa seluruh 36 event berhasil dikirim, dengan distribusi 13 event pada partisi 0, 12 event pada partisi 1, dan 11 event pada partisi 2.

- Kafka consumer: Script scripts/kafka_consumer.py membaca event menggunakan consumer group orders-validation-group, lalu menjalankan validasi. Hasilnya adalah 30 event valid dan 6 event ditolak. Hasil valid disimpan ke data/valid/orders_kafka_valid.parquet, sementara event yang ditolak disimpan ke data/rejected/orders_kafka_rejected.parquet dan file CSV.

- Consumer group dan offset: Consumer group mengelola progres pembacaan event melalui offset. Tiga partisi memungkinkan pekerjaan dibagi antarconsumer dalam group, walaupun jumlah consumer aktif menentukan bagaimana partisi ditugaskan. Offset yang telah diproses dapat di-commit agar consumer mengetahui posisi terakhir saat berjalan kembali.

- At-least-once dan idempotensi: Consumer menulis hasil sebelum melakukan commit offset. Pola ini mendukung pemrosesan at-least-once, tetapi memungkinkan suatu event diproses ulang jika program berhenti setelah penulisan dan sebelum commit. Karena itu, sink menerapkan deduplikasi berdasarkan identitas transaksi untuk membantu mencegah duplikasi hasil.

- Analisis: Eksperimen Kafka berhasil memperlihatkan alur event streaming, pembagian partisi, validasi di sisi consumer, dan penanganan event yang ditolak. Namun, at-least-once bukan jaminan exactly-once. Implementasi ini juga masih menggunakan satu broker lokal dan belum membuktikan jaminan exactly-once secara menyeluruh.

6. Benchmark Pandas dan Polars

- Tujuan benchmark: Benchmark dilakukan untuk membandingkan waktu pemrosesan Pandas dan Polars ketika melakukan agregasi data transaksi berdasarkan kategori produk. Pengujian melibatkan tiga pendekatan, yaitu Pandas eager, Polars eager, dan Polars lazy.

- Pandas eager: Pandas menjalankan operasi secara langsung ketika instruksi dieksekusi. Pendekatan ini mudah dipahami dan banyak digunakan untuk manipulasi data berbentuk tabel, tetapi performanya dapat berbeda tergantung ukuran dan karakteristik data.

- Polars eager: Polars menjalankan operasi secara langsung seperti Pandas, tetapi menggunakan mesin pemrosesan yang berbeda. Pada eksperimen ini, waktu pemrosesan Polars eager lebih rendah dibandingkan Pandas eager.

- Polars lazy: Polars lazy menyusun operasi sebagai rencana pemrosesan yang dapat dioptimalkan sebelum dijalankan. Pada pengujian, pendekatan ini memiliki rata-rata waktu paling rendah untuk dataset sumber yang digunakan.

Hasil benchmark dataset awal setelah 10 kali pengulangan:

Metode	Rata-rata waktu
Pandas Eager	0,008101 detik
Polars Eager	0,001927 detik
Polars Lazy	0,001804 detik

- Analisis hasil: Pada dataset yang hanya berisi 36 baris, Polars eager sekitar 4,2 kali lebih cepat dan Polars lazy sekitar 4,5 kali lebih cepat daripada Pandas eager berdasarkan rata-rata pengujian tersebut. Polars lazy sekitar 6,4% lebih cepat daripada Polars eager. Angka ini merupakan hasil eksperimen pada lingkungan yang digunakan, bukan jaminan bahwa Polars selalu lebih cepat untuk semua dataset.

7. Eksperimen distribusi data atau data skew

- Tujuan eksperimen: Pengujian skew dilakukan untuk mengetahui bagaimana distribusi kategori dapat berkaitan dengan waktu pemrosesan agregasi. Dua dataset dibuat dengan ukuran sama, masing-masing 200.000 baris dan 20 kategori, tetapi memiliki distribusi kategori yang berbeda.

- Dataset balanced: Data balanced mempunyai distribusi kategori yang relatif merata. Kategori terbesar mencakup sekitar 5,08% dari keseluruhan baris, sehingga tidak ada satu kategori yang mendominasi secara ekstrem.

- Dataset skewed: Data skewed memiliki satu kategori yang mendominasi sekitar 95,04% baris. Kondisi ini menggambarkan ketimpangan distribusi data yang dapat menjadi perhatian dalam sistem pemrosesan paralel dan operasi pengelompokan.

- Hasil benchmark:

Metode	Balanced	Skewed
Pandas Eager	0,079601 detik	0,064511 detik
Polars Eager	0,011365 detik	0,012172 detik
Polars Lazy	0,010239 detik	0,012443 detik

- Analisis hasil: Pada pengujian ini, Pandas justru lebih cepat pada dataset skewed, sedangkan Polars eager dan lazy sedikit lebih lambat. Hal tersebut menunjukkan bahwa distribusi data tidak selalu berbanding lurus dengan waktu pemrosesan. Perbedaan kecil dapat dipengaruhi kondisi lingkungan, overhead pembacaan Parquet, implementasi agregasi, dan karakteristik mesin. Karena itu, hasil ini tidak cukup untuk menyimpulkan bahwa skew selalu memperlambat seluruh metode.

8. Pembangunan data warehouse menggunakan DuckDB

- Tujuan warehouse: Setelah proses validasi, transaksi valid dimasukkan ke DuckDB sebagai dasar penyimpanan analitis. DuckDB dipilih karena dapat melakukan query SQL secara lokal dan terintegrasi dengan file Parquet, sehingga sesuai untuk proyek praktikum tanpa perlu menjalankan server database terpisah.

- Proses loading: Script warehouse/load_warehouse.py membaca data/valid/orders_valid.parquet dan memuatnya ke tabel raw_orders di database warehouse/ecommerce.duckdb. Tabel ini berisi 30 transaksi valid dengan 30 order_id unik.

- Analisis: Data warehouse menggunakan data yang sudah lolos validasi, bukan seluruh 36 baris sumber. Dengan demikian, keenam transaksi bermasalah tidak ikut masuk ke dalam analisis penjualan utama. Pemisahan ini menjaga konsistensi data yang digunakan pada tahapan transformasi selanjutnya.

9. Transformasi data menggunakan dbt

- Tujuan dbt: dbt digunakan untuk mengubah data mentah yang telah dimuat ke warehouse menjadi model analitis yang terstruktur. Transformasi ditulis menggunakan SQL sehingga setiap model dapat ditinjau, dijalankan kembali, dan diuji.

- Staging model: Model stg_orders membersihkan teks, menstandarkan status, mengonversi tipe data, dan membentuk kolom turunan seperti order_date, gross_amount, discount_amount, dan net_amount. Nilai penjualan bersih dihitung dengan memperhitungkan diskon transaksi.

- Dimension models: dim_customers merangkum pelanggan, waktu transaksi pertama dan terakhir, serta jumlah transaksi. dim_product_categories merangkum kategori produk, sedangkan dim_payment_methods merangkum metode pembayaran. Tiga dimensi ini membantu analisis dari sudut pandang pelanggan, kategori, dan pembayaran.

- Fact model: fact_orders menggabungkan transaksi staging dengan dimensi pelanggan, kategori produk, dan metode pembayaran. Model ini menjadi tabel fakta utama untuk analisis pada tingkat transaksi.

- Daily sales mart: daily_sales_mart merangkum data per tanggal, meliputi jumlah pesanan, jumlah pelanggan, kuantitas, penjualan bruto, diskon, penjualan bersih, serta jumlah pesanan berdasarkan status. Mart ini membuat metrik harian lebih mudah dianalisis tanpa harus menulis ulang query agregasi transaksi dari awal.

- Hasil eksekusi dbt: Perintah dbt run berhasil membuat enam model, terdiri dari satu staging view dan lima tabel. Perintah dbt test juga berhasil menjalankan 24 pengujian data tanpa kegagalan. Dokumentasi katalog berhasil dibuat menggunakan dbt docs generate, dan dokumentasi dapat ditampilkan melalui layanan dbt docs.

10. Hasil akhir analisis transaksi

- Jumlah data: Tabel raw_orders, stg_orders, dan fact_orders masing-masing memuat 30 transaksi. Dimensi pelanggan berisi 25 pelanggan, dimensi kategori berisi 4 kategori, dan dimensi metode pembayaran berisi 3 metode. daily_sales_mart menghasilkan 10 baris untuk periode 1–10 September 2026.

- Ringkasan nilai penjualan: Berdasarkan 30 transaksi valid yang masuk ke warehouse, hasil agregasi menunjukkan penjualan bruto sebesar 4.245,83, total diskon sebesar 297,37, dan penjualan bersih sebesar 3.948,48. Nilai tersebut mengikuti satuan moneter yang tersirat dalam dataset sumber; dataset tidak menetapkan mata uang secara eksplisit.

Transaksi sumber

36

Seluruh data awal

Transaksi valid

30

83,33% dari data sumber

Transaksi ditolak

6

16,67% dari data sumber

Pengujian dbt

24/24

Seluruh tes berhasil

- Perhitungan penjualan bersih: Secara konseptual, penjualan bruto dihitung dari kuantitas dikalikan harga satuan. Diskon dihitung dari penjualan bruto dikalikan persentase diskon. Penjualan bersih adalah penjualan bruto dikurangi diskon. Rumus ini diimplementasikan pada staging model dbt agar metrik dapat dihitung secara konsisten.

- Makna hasil bisnis: Mart penjualan harian memungkinkan pengguna melihat perubahan penjualan dari hari ke hari, membandingkan jumlah transaksi, serta mengevaluasi kontribusi diskon. Dimensi pelanggan, kategori produk, dan metode pembayaran juga menyediakan dasar untuk analisis yang lebih mendalam jika dibutuhkan.

11. Pengujian kualitas dan konsistensi

- Pengujian ingestion: Proses batch dijalankan dua kali untuk membuktikan bahwa data yang sama tidak menambah jumlah transaksi. Hasilnya, total tetap 36 baris dan tidak ada transaksi baru pada eksekusi kedua.

- Pengujian Kafka: Producer berhasil mengirim 36 event, sedangkan consumer memproses 36 event menjadi 30 data valid dan 6 data rejected. Hasil tersebut konsisten dengan jumlah data pada proses validasi sumber.

- Pengujian warehouse: Pemeriksaan DuckDB menunjukkan 30 transaksi valid dan 30 ID transaksi unik pada tabel transaksi utama. Hal ini menunjukkan bahwa data yang masuk ke warehouse sesuai dengan hasil validasi.

- Pengujian dbt: Sebanyak 24 tes berhasil, mencakup aturan seperti unique, not_null, dan relationships. Tes tersebut membantu mendeteksi ID yang berulang, kolom penting yang kosong, serta referensi dimensi yang tidak sesuai.

- Analisis keseluruhan: Rangkaian pengujian menunjukkan konsistensi jumlah data dari validasi hingga warehouse, sekaligus memverifikasi aturan integritas yang ditetapkan pada model dbt. Meskipun demikian, pengujian ini belum menggantikan pengujian skala besar, pemulihan broker, dan penanganan data terlambat pada sistem produksi.

12. Kesimpulan akhir

- Keberhasilan implementasi: Proyek berhasil membangun pipeline yang mencakup batch ingestion, retry, watermark, deduplikasi, validasi berbasis kontrak, event streaming Kafka, benchmark Pandas dan Polars, penyimpanan DuckDB, serta transformasi dan pengujian dbt. Artefak utama dan dokumentasi proyek telah dibuat.

- Temuan kualitas data: Dari 36 transaksi awal, enam transaksi ditolak karena melanggar aturan kualitas data. Proses validasi memastikan hanya 30 transaksi yang digunakan untuk analisis warehouse. Pemisahan data rejected juga membuat penyebab kesalahan dapat ditelusuri.

- Temuan performa: Polars eager dan lazy menunjukkan waktu rata-rata lebih rendah daripada Pandas pada dataset awal yang kecil. Namun, hasil eksperimen skew menunjukkan bahwa performa juga bergantung pada kondisi pengujian dan karakteristik data. Pemilihan framework sebaiknya didasarkan pada benchmark yang sesuai dengan kebutuhan nyata.

- Temuan data warehouse: dbt berhasil membangun enam model analitis dan menjalankan 24 tes tanpa kegagalan. Data mart menyediakan ringkasan penjualan harian yang dapat digunakan sebagai dasar analisis bisnis.

- Keterbatasan dan pengembangan: Proyek masih menggunakan dataset CSV sebagai simulasi API dan satu broker Kafka untuk demonstrasi lokal. Pengembangan selanjutnya dapat mencakup integrasi API sungguhan, penanganan late-arriving data, pengujian dataset yang lebih besar, penguatan mekanisme idempotensi, serta konfigurasi Kafka multi-broker untuk meningkatkan ketahanan sistem.

- Kesimpulan umum: Proyek ini menunjukkan bahwa pipeline data yang baik tidak hanya berfokus pada pengolahan data, tetapi juga membutuhkan kontrol kualitas, mekanisme pemulihan kegagalan, pencegahan duplikasi, pengujian performa, integritas warehouse, dan dokumentasi. Seluruh komponen tersebut bekerja bersama untuk menghasilkan data transaksi yang lebih konsisten dan siap digunakan untuk analisis.